

第7章: 書籍管理アプリ — 一覧表示と登録機能の実装

いよいよアプリの本体を作ります！この章では、**CRUD**（クラッド／create・read・update・delete の頭文字を取った言葉。データに対する基本4操作のこと）のうち、**R（Read：一覧表示）**と**C（Create：新規登録）**を実装します。

CRUD の4つの操作とは

文字	英語	日本語	例
C	Create	作成	新しい書籍を登録する
R	Read	読み取り	書籍の一覧を表示する／1冊の詳細を見る
U	Update	更新	書籍の評価やステータスを変更する
D	Delete	削除	不要になった書籍を消す

ほとんどのアプリ（SNS・ECサイト・メモアプリなど）は、結局この4操作の組み合わせでできています。この章ではまず **C** と **R** を作ります。

この章で作るもの

この章を終えると、以下の2つの画面が動くようになります。

画面	できること	CRUDのどれ？
書籍一覧ページ（トップページ）	Supabaseに保存された書籍をカード形式で表示する	R（Read：読み取り）
書籍登録ページ	フォームに入力して新しい書籍を登録する	C（Create：作成）

ユーザー体験の流れ

ユーザーの視点で、完成後の操作の流れを確認しましょう。

1. アプリを開く → **書籍一覧ページ**が表示される（登録済みの書籍がカード形式で並ぶ）
2. 「+ 新規登録」ボタンを押す → **登録フォーム**に移動する
3. タイトル・著者・評価などを入力して「登録する」を押す → データがSupabaseに保存される
4. 自動的に一覧ページに戻る → 今登録した書籍が一覧に追加されている

用語メモ: - **Server Component** (サーバーコンポーネント / server component) : サーバー側でHTMLを組み立ててからブラウザに送る部品。データベース接続などに向いている。 - **Client Component** (クライアントコンポーネント / client component) : ブラウザの中で動く部品。クリックや入力などインタラクティブな処理に向いている。 - **revalidation** (リバリデーション / 再検証) : 「キャッシュ済みのデータをもう一度サーバーから取り直す」こと。データ更新後に画面を最新化するため使う。 - **楽観的UI** (らっかんてきユーアイ / optimistic UI) : サーバーの返事を待たずに「成功したつもり」で画面を先に更新するUI手法。体感速度を上げる代わりに失敗時のロールバックが必要になる。この章では使わないが、第8章以降で登場する。

作成するコンポーネント（部品）一覧

コンポーネント名	役割	なぜ分けるのか
StatusBadge	「読書中」「読了」などのステータスを色付きバッジで表示	複数の画面で使い回すため
RatingStars	1～5の評価を星マーク（★）で表示	一覧・詳細の両方で使うため
BookCard	1冊分の書籍情報をカード型で表示	一覧画面でN冊分繰り返し表示するため
BookList	BookCardを並べて表示するコンテナ（入れ物）	レイアウトの責任を分離するため
BookForm	書籍の登録/編集に使うフォーム	登録と編集で同じフォームを使い回すため
LoadingSpinner	データ読み込み中の表示	ユーザーに「読み込み中です」と伝えるため

なぜコンポーネントを分けるの？ 1つの巨大なファイルに全てを書くこともできますが、部品に分けることで「見通しが良くなる」「バグを見つけやすくなる」「同じ部品を使い回せる」というメリットがあります。これは**コンポーネント設計**と呼ばれ、React開発の重要な考え方です。

目次

- 0. 前提知識: 非同期処理 (async / await) の超基礎
- 1. 一覧表示機能の実装
- 2. ローディング状態
- 3. エラーハンドリング
- 4. 登録機能の実装
- 5. コード解説
- 6. 動作確認手順
- 7. トラブルシューティング

0. 前提知識: 非同期処理（async / await）の超基礎

この章のコードでは `async`（エイシンク）と `await`（アウェイト）が頻出します。これは「時間がかかる処理を待つ」ための仕組みです。Supabaseに「データを取ってきて」とお願いする処理は、結果が返ってくるまで数十～数百ミリ秒かかります（インターネット越しに別のサーバーへ問い合わせるので、CPU内部で完結する計算より格段に遅い）。その「待ち時間」を扱うのが `async/await` です。

読み方と意味: - **synchronous**（シンクロナス／同期）：「時間軸が揃っている」という意味。前の処理が終わってから次の処理に進む。 - **asynchronous**（エイシンクロナス／非同期）：「時間軸が揃っていない」。前の処理の終了を待たずに次の処理が始まることもある。

0.1 同期と非同期の違い

```
// 同期処理 (synchronous) : 上から下へ順番に実行され、即座に結果が返る
const sum = 1 + 2;           // この計算はCPU内部だけで完結するのでマイクロ秒で終わる
console.log(sum);           // 3 と表示される。1 + 2 の結果がすでに確定しているため。

// 非同期処理 (asynchronous) : 結果が返るのに時間がかかる
const data = supabase.from("books").select("*"); // ネット越しのリクエスト。返事が来るまで時間がかかる。
// ❌ await を付けないので、data には「答え」ではなく「答えが入る約束 (Promise)」が代入される。
```

0.2 Promise って何？

非同期処理の「結果が返ってくる予定の入れ物」のことを **Promise**（プロミス、約束／英語の "promise" は『約束』）と言います。

Promise は3つの状態を持ちます。 - **pending**（ペンディング／保留中）：まだ結果が出ていない。 - **fulfilled**（フルフィルド／成功）：結果（値）が確定した。 - **rejected**（リジエクトッド／失敗）：エラーで失敗した。

今すぐ返せる値： — 1 + 2 — 即 3 が返る
非同期の値： — DB問い合わせ — ⌚ Promise が返る — 数百ms後に中身が確定

0.3 async / await で「待つ」

`await` を関数呼び出しの前に書くと、「Promiseの中身が確定するまでこの行で待ってね」という意味になります。

`await` を使う関数には **必ず** `async` を付ける必要があります。`async` の付いていない関数の中で `await` を書くと、TypeScript / JavaScript の文法エラーになります。

```
// ❌ await なし: data には Promise オブジェクトが入る
const fetchBooks = () => {
  const data = supabase.from("books").select("*");
  console.log(data);
  // 普通関数 (async が無い)
  // Promise が data に入る (中身はまだ無い)
  // Promise { <pending> } と表示される (ペンディング状態)
};

// ✅ async/await あり: data に実際のデータが入る
const fetchBooks = async () => {
  const { data } = await supabase.from("books").select("*");
  // async を付けたので await が使える関数になる
  // await により Promise の中身が確定するまで停止 → 確定した結果から data プロパティを取り出す (分割代入)
  console.log(data);
  // [{ id: 1, title: "..."}] のような配列が出る
};
```

▼ 概念図:

```
async function fetchBooks() {
  const { data } = await supabase.from("books").select("*");
  //      └─ ここで「結果が来るまで待つ」 (pendingからfulfilledへ変わる瞬間)
  console.log(data); // ← 結果が来てからこの行が実行される
}
```

重要なポイント: `await` を付けた関数を呼び出した瞬間、その関数自体も Promise を返します。つまり「`async`関数は常に Promise を返す」というルールがあります。`async function f() { return 1; }` を呼ぶと、戻り値は数値の 1 ではなく、中身が 1 の Promise になります。

0.4 try / catch でエラーを捕まえる

非同期処理は「サーバーが落ちている」「ネットワーク切断」「権限が無い」などで失敗することがあります。`try / catch` で失敗を捕まえます。

```
const fetchBooks = async () => { // async関数の定義
  try { // この中で起きたエラーは下の catch に飛ぶ
    const { data, error } = await supabase.from("books").select("*"); // 非同期処理をawait で待つ
    if (error) throw error; // Supabaseは「成功でも例外を投げない」設計。errorプロパティを自分で投げ直す。
    console.log("成功:", data); // 例外が出なかった=成功。data を使える。
  } catch (err) { // try ブロック内で throw された値や、await中の例外がここに来る
    console.error("失敗:", err); // 開発者ツール（コンソール）に赤字でエラーを出す
  }
};
```

▼ 出力イメージ:

成功時:

```
成功: [ { id: 1, title: "...", ... }, { id: 2, ... } ]
```

失敗時（テーブル名を間違えた場合など）:

```
失敗: { code: '42P01', message: 'relation "bookz" does not exist' }
```

本書での使い方: Supabaseの `.from()` などはすべて非同期なので、`async` 関数の中で `await` を付けて呼びます。エラーは `try/catch` で捕まえ、画面に「エラーが発生しました」と表示します。

1. 一覧表示機能の実装

一覧表示は、データベースに保存された書籍をカード形式で並べて表示する機能です。以下のコンポーネント構成で実装します。

```
app/page.tsx (トップページ / Server Component)
├── BookList (書籍一覧)
│   └── BookCard (書籍カード) × N冊分
│       ├── StatusBadge (ステータスバッジ)
│       └── RatingStars (星評価)
```

小さなコンポーネントから順に作っていきましょう。

1a. StatusBadge コンポーネント

ファイル: `components/StatusBadge.tsx`

書籍のステータス（読書中・読了・読みたい）をバッジ形式で表示するコンポーネントです。ステータスの値に応じて背景色とテキストが変わります。

▼ このコードがやること（先に日本語で）：「読書中」「読了」「読みたい」というステータスを、色付きの小さなラベル（バッジ）に変換して表示する部品を作ります。カギは「ステータスごとの色や文字を1つの辞書（オブジェクト）にまとめておき、本体はそこから引くだけ」という作り。こうすると `if` 文で分岐を書かずに済み、ステータスを増やすときも辞書に1行足すだけで済みます。初心者は「渡された `status` の値で見た目が変わる」という1点を押さえれば十分です（細かい仕組みはコード内のコメントで説明します）。

```
// =====
// ファイルパス: src/components/StatusBadge.tsx
// 役割      : 書籍のステータス（読書中／読了／読みたい）を、
//            色付きの小さなバッジとして画面に表示するための部品（コンポーネント）
// 親コンポーネント: BookCard が使う
// =====

/**
 * StatusBadge - 書籍の読書ステータスをバッジ形式で表示するコンポーネント
 *
 * 表示されるバッジの種類:
 * - reading      : 「読書中」 （青色）
 * - completed    : 「読了」   （緑色）
 * - want_to_read : 「読みたい」（黄色）
 *
 * 受け取るデータ:
 * - status: 上記3種類のうちどれか1つ（文字列）
 *
 * 出力（画面に出るもの）:
 * - 角丸の小さなラベル（spanタグ）。文字色と背景色がステータスごとに違う。
 */

// -----
// (1) 「ステータスの型」を定義する
// -----
// TypeScript の「ユニオン型 (|)」を使って
// 「BookStatus は文字列だけど、3つのうちのどれか」というルールを作る。
//
// こう書くことで、誤って "reading2" など typo のある文字列を渡したときに
// VS Code がコードを書いている最中に赤線で警告してくれる。
// 「export」を付けているので、ほかのファイル（BookCardなど）からも import して使える。
export type BookStatus = "reading" | "completed" | "want_to_read";

// -----
// (2) このコンポーネントが「親から受け取るプロパティ (Props)」の形を定義
// -----
// Reactでは、親コンポーネントから子コンポーネントに渡すデータを「Props」と呼ぶ。
// ここでは「status」というキーで BookStatus 型の値を1つ受け取る、と宣言している。
//
// 例: <StatusBadge status="reading" /> ← こう呼び出される。
type StatusBadgeProps = {
  status: BookStatus;
};

// -----
// (3) ステータスごとの表示設定をオブジェクトでまとめる
// -----
// もし if文 や switch文 で「reading の場合は青、completed の場合は緑...」と
// 書くと、新しいステータスを追加するたびに分岐を増やす必要があり保守が大変。
```

```

//
// そこで「ステータス → { 表示文字, 背景色, 文字色 }」という辞書（オブジェクト）を
// 作っておくと、コンポーネント本体はこの辞書を引くだけで済む。
//
// `Record<BookStatus, ...>` は「キーは BookStatus のどれか、値は { ... }」という
// TypeScriptの記法で、辞書全体の型を保証する。
//
// `bg-blue-100` などは Tailwind CSS のクラス名。
//   bg-          = background-color
//   blue-100     = 100段階で薄めの青（数字が大きいほど濃い）
//   text-blue-800 = テキストカラーを濃い青に
const statusConfig: Record<
  BookStatus,
  { label: string; bgColor: string; textColor: string }
> = {
  reading: {
    label: "読書中",          // バッジに表示する日本語
    bgColor: "bg-blue-100",    // 背景: 薄い青
    textColor: "text-blue-800", // 文字: 濃い青
  },
  completed: {
    label: "読了",
    bgColor: "bg-green-100",    // 背景: 薄い緑（完了感のある色）
    textColor: "text-green-800",
  },
  want_to_read: {
    label: "読みたい",
    bgColor: "bg-yellow-100",   // 背景: 薄い黄
    textColor: "text-yellow-800",
  },
};

// -----
// (4) コンポーネント本体
// -----
// `export default` で「このファイルの主役はこの関数だよ」と宣言。
// 関数の引数を `{ status }` と書いているのは「分割代入」と呼ばれる JS の文法。
//   - 引数として渡されたオブジェクト (StatusBadgeProps型) から
//     `status` という名前のプロパティだけを取り出す。
//   - 同等の書き方:
//       function StatusBadge(props: StatusBadgeProps) {
//         const status = props.status;
//         ...
//       }
export default function StatusBadge({ status }: StatusBadgeProps) {
  // (4-1) 渡された status をキーにして、設定オブジェクトを引く。
  //
  // ?? は「Nullish Coalescing 演算子」と呼ばれ、
  // 左辺が null か undefined の場合に右辺を使うという意味。
  // 万が一 statusConfig に存在しないキー（不正な値）が来た場合のための保険。

```



```

const config = statusConfig[status] ?? {
  label: "不明",
  bgColor: "bg-gray-100",
  textColor: "text-gray-800",
};

// (4-2) JSX を返す。これが画面に描かれるHTML（厳密にはReactが生成するDOM）。
//
// <span> はインライン要素（行の中で使う小さな箱）。
// className に Tailwind のクラスを並べることで、CSSを別ファイルに書かずに
// スタイルを当てられる。
//
// ${...} の部分はテンプレートリテラル。
// configの値（変数）を文字列の中に埋め込んでいる。
return (
  <span
    className={`
      inline-flex items-center /* 中身（テキスト）を上下中央に揃える */
      px-2.5 py-0.5 /* 内側の余白（padding x方向 / y方向）*/
      rounded-full /* 角を完全に丸めてカプセル型に */
      text-xs font-medium /* 文字サイズを小さく、太さは中ぐらい */
      ${config.bgColor} /* 背景色（ステータスにより変わる） */
      ${config.textColor} /* 文字色（ステータスにより変わる） */
    `}
    >
    {/* {config.label} はJSXで「JS式を埋め込む」記法。
      ここでは "読書中" などの文字列が出力される */}
    {config.label}
  </span>
);
}

// =====
// この StatusBadge.tsx を <StatusBadge status="reading" /> のように書くと:
//
// 出力されるHTML（概念）:
// <span class="inline-flex ... bg-blue-100 text-blue-800">読書中</span>
//
// 画面の見た目:
//
// | 読書中 | ← 青背景・青文字の角丸ラベル
//
// =====

```

▼ どこから呼び出されるの？

このコンポーネントは、後で出てくる `BookCard.tsx` の中から次のように使われます。

```
import StatusBadge from "@/components/StatusBadge";

// ...
<StatusBadge status={book.status} />
// 例: book.status が "reading" のとき、青いバッジが描画される
```

画面にはこのように表示されます:

StatusBadge は小さな角丸のバッジとして表示されます。

- 「読書中」の場合: 薄い青色の背景に、濃い青色のテキストで「読書中」と表示されます。見た目は **[読書中]** のような角丸の小さなラベルです。
- 「読了」の場合: 薄い緑色の背景に、濃い緑色のテキストで「読了」と表示されます。完了を連想させる緑色です。
- 「読みたい」の場合: 薄い黄色の背景に、濃い黄色（オレンジ寄り）のテキストで「読みたい」と表示されます。「まだ読んでいない＝これから」を連想させる色です。

いずれも **text-xs**（12px 程度）の小さめのフォントサイズで、**rounded-full** により完全な角丸（カプセル型）になります。カードの中に配置すると、ステータスがひと目で分かるアクセントになります。

1b. RatingStars コンポーネント

ファイル: **components/RatingStars.tsx**

書籍の評価（1～5）を星マーク（★☆）で視覚的に表示するコンポーネントです。

▼ このコードがやること（先に日本語で）: 「3」のような数字の評価を、★★★★☆ という星の見た目に変換して表示する部品を作ります。やり方は「1から5までの星を並べ、評価値以下の星は黄色で塗り、それより大きい星はグレーにする」だけです。評価が **null**（未入力）のときは星ではなく「評価なし」と出す分岐も用意します。初心者は「数字を見ただけに変換しているだけ」と捉えればOKです（星を並べる **map** や色分けの三項演算子はコード内コメントで解説します）。

```
// components/RatingStars.tsx ← このファイルのパス（コメントとして書いておくと分かりやすい）

/**
 * RatingStars - 1～5の評価を星マークで表示するコンポーネント
 *
 * 例: rating=3 の場合 → ★★★☆☆
 *
 * rating が null の場合は「評価なし」と表示する。
 * これは「評価が未入力の本」を表現できるようにするため。
 */

// Props（プロパティ：親から渡される値）の型を定義。
// rating は数値、もしくは null（評価がないことを表す）のどちらかを受け取る。
// 「number | null」は TypeScript の「ユニオン型」で、「AかBどちらか」という意味。
type RatingStarsProps = {
  rating: number | null;
};

// export default = このファイルの主役の関数。他のファイルから import RatingStars from
// "./RatingStars" で読み込める。
// 引数で { rating } と分割代入しているので、props.rating ではなく rating だけで使える。
export default function RatingStars({ rating }: RatingStarsProps) {
  // 評価がない場合はテキストで表示する。
  // === は「型まで含めて完全一致」する比較演算子。==（緩い比較）よりも安全なので必ずこちらを使う。
  // null と undefined はどちらも「値が無い」ことを表すが、別物として扱われるため両方チェックしている。
  if (rating === null || rating === undefined) {
    return (
      // 早期 return で、以降の処理を実行せずにこの JSX を返して終わる。
      // text-sm = 小さな文字、text-gray-400 = 薄いグレー（控えめなトーン）。
      <span className="text-sm text-gray-400">
        評価なし
      </span>
    );
  }

  // 1～5 の範囲に収める（データベースの制約と合わせる）。
  // Math.max(1, rating) = rating と 1 の大きい方 → 1未満なら 1 に押し上げる
  // Math.min(5, 上記結果) = 上記結果と 5 の小さい方 → 5を超えるなら 5 に押し戻す
  // 結果として「1以上5以下」に強制される。これを「クランプ (clamp、挟み込み)」と呼ぶ。
  const clampedRating = Math.min(5, Math.max(1, rating));

  return (
    // flex = 横並び、items-center = 縦方向中央揃え、gap-0.5 = 子要素間の隙間。
    // aria-label はスクリーンリーダー（視覚障害者向け読み上げ機能）に伝える説明文。
    // `${clampedRating}点` はテンプレートリテラル：バッククォート ` で囲み、`${}` で変数を埋め込める。
    <div className="flex items-center gap-0.5" aria-label={`評価: ${clampedRating}点`} >
      { /* 星を5つ並べる。rating以下のインデックスは塗りつぶし、それ以外は空の星。 */ }
    </div>
  );
}

```

```

[1, 2, 3, 4, 5].map(...) は「配列の各要素に関数を適用して新しい配列を作る」メソッド。
ここでは数値1~5に対して <span> を作って配列にしている。 */}
{[1, 2, 3, 4, 5].map((star) => (
  <span
    // key は React が「どの要素がどれか」を識別するための必須プロパティ。
    // map で要素を並べる時は必ず key を付ける（付けないと警告が出る）。
    key={star}
    // 三項演算子（条件 ? A : B）で「条件が真なら A、偽なら B」を選ぶ。
    // star（1~5）が現在の評価値以下なら「塗りつぶし（黄色）」、それより大きいなら「空（グ
一）」。
    className={`text-lg ${
      star <= clampedRating
        ? "text-yellow-400" // 塗りつぶしの星（黄色）
        : "text-gray-300" // 空の星（グレー）
      }`}
    >
      ★ {/* 実は同じ「★」を出している。色だけクラスで切り替えることで塗りつぶし/空を表現。
*/}
    </span>
  )]}
  {/* 数値も併記する（スクリーンリーダーや視認性のため）。
    ml-1 は left margin（左の外側余白）。星の右に少し離して数値を置く。 */}
  <span className="ml-1 text-sm text-gray-600">
    ({clampedRating})
  </span>
</div>
);
}

```

画面にはこのように表示されます:

- rating=4 の場合: ★★★★★☆ (4) — 黄色い星4つとグレーの星1つ、その横に小さく (4) と数値が表示されます。
- rating=null の場合: グレーの文字で 評価なし と表示されます。

1c. BookCard コンポーネント

ファイル: **components/BookCard.tsx**

1冊分の書籍情報をカード形式で表示するコンポーネントです。上で作った StatusBadge と RatingStars を組み合わせて使います。

▼ このコードがやること（先に日本語で）：書籍1冊分の情報（タイトル・著者・出版社・ステータス・評価など）を、1枚のカードにまとめて表示する部品を作ります。ポイントは、さきほど作った `StatusBadge` と `RatingStars` を子として組み合わせて使うこと——小さな部品を寄せ集めて大きな部品を作る考え方（コンポジション）の実例です。さらにカード全体を `<Link>` で囲み、どこをクリックしても詳細ページへ飛ぶようにします。初心者は「1冊分のデータを受け取り、カードの形に並べているだけ」と押さえれば十分です（各部分の意味はコード内コメントにあります）。

```
// =====
// ファイルパス: src/components/BookCard.tsx
// 役割      : 1冊分の書籍をカード形式で表示する。クリックで詳細ページへ遷移。
// 親        : BookList コンポーネントから1冊ずつ渡されて表示される
// -----
// このコンポーネントは、これまでに作った StatusBadge と RatingStars を
// 子として組み合わせて使う「コンポジション（合成）」の好例。
// =====

// (1) 必要なものを import
//   Next.js の Link: クライアントサイド遷移用のリンク要素（フルリロードしない）
//   ./StatusBadge  : 同フォルダの StatusBadge.tsx（拡張子は省略可能）
//   `type BookStatus` : 値ではなく「型」だけを取り込む記法
import Link from "next/link";
import StatusBadge, { type BookStatus } from "./StatusBadge";
import RatingStars from "./RatingStars";

/**
 * BookCard - 書籍1冊分の情報をカード形式で表示するコンポーネント
 *
 * 表示する情報:
 * - タイトル
 * - 著者
 * - 出版社（あれば）
 * - ステータスバッジ
 * - 評価（星）
 *
 * カード全体が <Link> で囲まれており、どこをクリックしても /books/{id} へ遷移する。
 */

// (2) Book 型の定義
//   データベースの books テーブルの1行に対応する型。
//   publisher など null OK のカラムは「string | null」のユニオン型で表す。
//   export しているので他ファイルから import { type Book } で使える。
export type Book = {
  id: string;           // UUID（主キー）
  title: string;        // 必須
  author: string;       // 必須
  publisher: string | null; // 任意
  published_date: string | null; // ISO日付文字列（"2024-01-15" など）
  rating: number | null; // 1～5 または null
  status: BookStatus;   // "reading" | "completed" | "want_to_read"
  memo: string | null;  // 感想メモ
  created_at: string;   // 作成日時（自動）
  updated_at: string;   // 更新日時（自動）
};

// (3) Props の型
//   親 (BookList) から book を1つ受け取る、というだけの単純な型。
```

```

type BookCardProps = {
  book: Book;
};

// (4) コンポーネント本体
//   export default で「このファイルの主演」を1つだけ宣言。
//   関数引数 ({ book }: BookCardProps) は分割代入 + 型注釈。
export default function BookCard({ book }: BookCardProps) {
  return (
    // (5) Link はカード全体を「クリック可能なリンク」にする要素
    //   href={`\books/${book.id}`} のテンプレートリテラルで動的にURLを組み立てる
    //   /books/123e4567-e89b-12d3-a456-426614174000 のような遷移先になる
    //
    //   className のクラス文字列の意味 (Tailwind CSS) :
    //     block                ← <Link> (普段はインライン) をブロック要素にする
    //     bg-white             ← 背景色を白に
    //     rounded-lg           ← 角丸 (大きめ)
    //     shadow-md            ← 中ぐらいのドロップシャドウ
    //     hover:shadow-lg      ← マウスを乗せたら影を強くする
    //     transition-shadow    ← 影の変化を滑らかにアニメーション
    //     duration-200         ← アニメーション時間 200ms
    //     p-6                  ← 全方向の内側余白 24px
    //     border               ← 1px の枠線を表示
    //     border-gray-200      ← 枠線の色
    //     hover:border-blue-300 ← ホバー時に枠線の色を変える
    <Link
      href={`\books/${book.id}`}
      className="
        block
        bg-white
        rounded-lg
        shadow-md
        hover:shadow-lg
        transition-shadow
        duration-200
        p-6
        border
        border-gray-200
        hover:border-blue-300
      "
    >
    { /*
      (6) 1行目: タイトル と StatusBadge を左右に配置
      flex                ← フレックスレイアウト
      items-start          ← 上端揃え (タイトルが2行になっても揃う)
      justify-between      ← 左右の端に寄せる
      gap-2                ← 子要素間の隙間
      mb-3                 ← 下方向の外側余白
    */ }
    <div className="flex items-start justify-between gap-2 mb-3">

```

```

    {/*
      line-clamp-2 = テキストが長くて2行で「...」省略する Tailwind 補助クラス
      flex-1      = 余白を最大限取る (バッジに押し出されないようにする)
    */}
    <h2 className="text-lg font-bold text-gray-900 line-clamp-2 flex-1">
      {book.title}
    </h2>

    {/*
      (7) 子コンポーネント StatusBadge を呼び出す。Props として status を渡す
      <StatusBadge status="reading" /> のように展開される。
    */}
    <StatusBadge status={book.status} />
  </div>

  {/* (8) 著者名 */}
  <p className="text-sm text-gray-600 mb-1">
    <span className="font-medium text-gray-500">著者:</span>{" "}
    {book.author}
  </p>

  {/*
    (9) 「book.publisher && ...」は短絡評価。
    publisher が null/空 のときは何も表示せず、値があるときだけ <p> を描画。
  */}
  {book.publisher && (
    <p className="text-sm text-gray-600 mb-1">
      <span className="font-medium text-gray-500">出版社:</span>{" "}
      {book.publisher}
    </p>
  )}

  {/* (10) 出版日 (null チェックは publisher と同じパターン) */}
  {book.published_date && (
    <p className="text-sm text-gray-600 mb-3">
      <span className="font-medium text-gray-500">出版日:</span>{" "}
      {book.published_date}
    </p>
  )}

  {/*
    (11) 評価 (星)
    mt-3 pt-3 border-t border-gray-100 で
    「カード本体と評価エリアの間に薄い区切り線を入れる」効果。
  */}
  <div className="mt-3 pt-3 border-t border-gray-100">
    <RatingStars rating={book.rating} />
  </div>

  {/* (12) メモが存在すれば、最大2行までプレビュー表示 */}

```



```

    {book.memo && (
      <p className="mt-2 text-xs text-gray-400 line-clamp-2">
        {book.memo}
      </p>
    )}
  </Link>
);
}

```

1枚のカードはこのように表示されます:

カードは白い背景（`bg-white`）に薄いグレーのボーダー（`border-gray-200`）と影（`shadow-md`）がついた角丸の矩形です。マウスカーソルを乗せると影が濃くなり（`hover:shadow-lg`）、ボーダーが薄い青色（`hover:border-blue-300`）に変わります。これにより「クリックできる」ことが視覚的に伝わります。

カードの内部構成は上から順に:

1. 1行目（タイトル行）: 左にタイトルが太字で表示され、右端にステータスバッジが配置されます。タイトルが長い場合は2行まで表示され、それ以降は `...` で省略されます（`line-clamp-2`）。
2. 2行目: 「著者: ○○」のように著者名が小さめのフォントで表示されます。
3. 3行目（出版社がある場合）: 「出版社: ○○」と表示されます。
4. 4行目（出版日がある場合）: 「出版日: 2024-01-15」のように表示されます。
5. 区切り線の下: 薄いグレーの区切り線の下に星評価（例: ★★★★★（4））が表示されます。
6. 最下部（メモがある場合）: メモの先頭部分が薄いグレーで2行まで表示されます。

カード全体が `<Link>` で囲まれているため、どこをクリックしても `/books/[id]` の詳細ページに遷移します。

1d. BookList コンポーネント

ファイル: `components/BookList.tsx`

BookCard を並べて表示するコンポーネントです。書籍が0冊の場合のメッセージも含みます。

▼ このコードがやること（先に日本語で）: 書籍の配列を受け取り、**BookCard** を冊数分だけ並べて一覧表示する入れ物（コンテナ）を作ります。カギは2つ——① `map` で「配列の各書籍を1枚のカードに変換」して並べること、② 書籍が0冊のときは空状態（`empty state`）として案内メッセージと登録ボタンを出す分岐を持つこと。並べ方は CSS Grid で、画面幅に応じて1〜3列に自動で変わります。初心者は「カードを並べる係」と「0冊のときの案内係」の2役だと押さえればOKです（詳細はコード内コメントにあります）。

```
// components/BookList.tsx ← ファイルパス

// 同フォルダの BookCard を読み込む。Book 型も同時に取り込む（コンポーネントと型を同一ファイルから
// export しているのは効率のため）。
// 「type」キーワードを付けると「型だけインポート」になり、ビルド時に消えるので JS バンドルサイズが
// 小さくなる。
import BookCard, { type Book } from "./BookCard";

/**
 * BookList - 書籍一覧をグリッドレイアウトで表示するコンポーネント
 *
 * - 書籍が1冊以上ある場合: カードをグリッド状に並べる
 * - 書籍が0冊の場合: 「書籍が登録されていません」メッセージを表示
 *
 * レスポンシブ対応 (responsive: 画面幅で見た目を変える):
 * - スマホ (デフォルト): 1列
 * - タブレット (md = medium breakpoint: 768px~): 2列
 * - デスクトップ (lg = large breakpoint: 1024px~): 3列
 */

// Props 型: 親 (page.tsx など) から books 配列を受け取るだけのシンプルな型。
// 「Book[]」は「Book型を要素とする配列」という意味。
type BookListProps = {
  books: Book[];
};

export default function BookList({ books }: BookListProps) {
  // 書籍が0冊の場合の「空状態 (empty state)」表示。
  // books.length は配列の要素数。0なら登録なし。
  if (books.length === 0) {
    return (
      // text-center で中央寄せ、py-16 で上下に大きめの余白を取って「主役感」を出す。
      <div className="text-center py-16">
        { /* 大きなアイコン的テキスト。絵文字を巨大表示することで視覚的なアクセントにする。 */ }
        <p className="text-6xl mb-4">📖</p>
        <h2 className="text-xl font-bold text-gray-700 mb-2">
          書籍が登録されていません
        </h2>
        <p className="text-gray-500 mb-6">
          「新規登録」ボタンから最初の書籍を登録しましょう。
        </p>
        { /* <a> は通常のリンク。<Link> でも書けるが、ここでは Next.js のインポートを増やさないと
        普通の a タグにしている。
          ※本格運用では <Link> を使った方がページ間遷移が高速になる（プリフェッチ機能のため）。
        */ }
        <a
          href="/books/new"
          className="
            inline-block
            /* aタグ（普段インライン）をブロック寄りにして余白を効かせ

```

```

る */
    bg-blue-600          /* 背景：濃い青 */
    text-white           /* 文字色：白 */
    px-6 py-3           /* 内側余白：横6 / 縦3 (Tailwindの単位は4px基準) */
    rounded-lg           /* 角丸 */
    font-medium          /* 文字の太さ：中 */
    hover:bg-blue-700    /* マウスホバー時にもう一段濃い青へ */
    transition-colors    /* 色の変化を滑らかに */
    duration-200         /* アニメーション時間 200ms */
  "
  >
    最初の書籍を登録する
  </a>
</div>
);
}

// 書籍が1冊以上ある場合の表示 (早期 return しなかったので、ここに到達)。
return (
  <div
    // grid と grid-cols-X で CSS Grid Layout を使う。
    // grid-cols-1       : デフォルト (スマホ) は1列
    // md:grid-cols-2     : 中サイズ画面以上では2列に切り替え
    // lg:grid-cols-3     : 大画面では3列に切り替え
    // gap-6              : セル間の隙間 (24px)
    className="
      grid
      grid-cols-1
      md:grid-cols-2
      lg:grid-cols-3
      gap-6
    "
  >
    {/* books 配列を map で1冊ずつ BookCard に変換して並べる。
      key={book.id} は React の必須要件: 「どの要素がどれか」を識別するための一意キー。
      book={book} は Props として book オブジェクトを子に渡している ("book" という名前で受け取る)。 */}
    {books.map((book) => (
      <BookCard key={book.id} book={book} />
    ))}
  </div>
);
}

```

表示の説明:

- 書籍がある場合: CSS Grid を使って、画面幅に応じて1〜3列にカードが並びます。各カードの間は `gap-6` (24px) の余白があります。

- 書籍がない場合: 画面中央に本の絵文字、「書籍が登録されていません」というメッセージ、そして新規登録ページへのリンクボタンが表示されます。

1e. トップページ

ファイル: `app/page.tsx`

Supabase からデータを取得して BookList に渡すトップページです。Next.js の **Server Component** として実装します。

▼ このコードがやること（先に日本語で）：アプリのトップページとして、Supabase から書籍一覧を取ってきて、**BookList** に渡して画面に表示する部分を作ります。カギは、これが **Server Component**（サーバー側で動く部品）であること——だから関数に `async` を付け、`await` でデータ取得を待ってから、完成したHTMLをブラウザに送れます。データベースの秘密情報がブラウザに漏れず、初回表示も速い、というメリットがあります。初心者は「サーバーでデータを取り、それを一覧部品に渡しているだけ」と押さえれば十分です（取得処理やエラー処理の詳細はコード内コメントにあります）。

```
// app/page.tsx ← Next.js App Router の規約で「ルート (/) に対応するページ」になる

// Next.js が提供する Link コンポーネント。<a> よりも遷移が速い（プリフェッチ機能あり）。
import Link from "next/link";
// サーバー用 Supabase クライアント。cookies などの「サーバーしか知らない情報」を扱えるバージョン。
import { createClient } from "@lib/supabase/server";
// 自作の一覧表示コンポーネント。「@/」は tsconfig.json で設定した「プロジェクトルート」のエイリアス。
import BookList from "@components/BookList";
// Book 型だけを取り込む。値ではなく型なので「type」キーワード付き。
import { type Book } from "@components/BookCard";

/**
 * トップページ (Server Component)
 *
 * このコンポーネントは Server Component として動作する。
 * つまり、サーバー側で Supabase からデータを取得し、
 * HTML をレンダリングしてからクライアントに送信する。
 *
 * "use client" を書いていない = Server Component (Next.js App Router のデフォルト)
 *
 * 重要な特徴:
 * - 関数定義に async が付いている → 内部で await が使える
 * - サーバー上で動くので、Supabase の秘密鍵 (service role 等) も安全に使える
 * - JavaScript としてブラウザに送信されないため、バンドルサイズが小さい
 * - useState や onClick などの「インタラクティブな機能」は使えない（必要なら子要素に Client Component を置く）
 */
export default async function HomePage() {
  // Supabase クライアントを作成（サーバー用）。
  // この createClient は内部で cookies() を呼ぶので async (await が必要)。
  const supabase = await createClient();

  // books テーブルから全件取得（作成日の降順＝新しい順）。
  //   .from("books")           : 操作対象テーブルを指定
  //   .select("*")             : 全カラムを取得 (SELECT * 相当)
  //   .order("created_at", { ascending: false }) : created_at で並び替え、false なので降順（新しい順）
  // await で結果を待つと、{ data, error } という形のオブジェクトが返る。
  // 「data: books」は「data プロパティを books という別名で取り出す」分割代入のリネーム記法。
  const { data: books, error } = await supabase
    .from("books")
    .select("*")
    .order("created_at", { ascending: false });

  // エラーが発生した場合の処理。
  // Supabase は通常エラーを throw せず、戻り値の error プロパティに格納する設計。
  if (error) {
    // console.error はサーバーのターミナルにログを出す（ブラウザではなくサーバー側に表示される）。
  }
}
```

```

    console.error("書籍の取得に失敗しました:", error.message);
    // エラー時でも画面は表示する（空の一覧として）。
    // 本番アプリでは throw error して error.tsx で受け止める設計にすることもある。
  }

  // data が null の場合は空配列にする。
  // 「??」 (Nullish Coalescing 演算子) は左辺が null か undefined の時だけ右辺を使う。
  // これにより books が null でも以降のコードで安心して .length などと呼べる。
  const bookList: Book[] = books ?? [];

  return (
    // min-h-screen = 最低でも画面の高さ分 (h = height、screen = ビューポート1画面)
    // bg-gray-50 = 非常に薄いグレー背景 (コンテンツの白を引き立てる)
    <div className="min-h-screen bg-gray-50">
      {/* ===== ハッダー ===== */}
      {/* <header> は意味的なHTML要素 (セマンティクス)。ページの上部「ナビ・タイトル」帯に使う。
        shadow-sm = 薄い影、border-b = 下方向の枠線 */}
      <header className="bg-white shadow-sm border-b border-gray-200">
        {/* max-w-7xl = 最大幅 1280px、mx-auto = 左右の余白を自動 (中央寄せ)。
          px-4 sm:px-6 lg:px-8 = 横の内側余白を画面幅で段階的に変更 (レスポンス)。 */}
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-6">
          {/* flex で横並び、items-center で縦中央、justify-between で「左右両端に寄せる」。
            */}

          <div className="flex items-center justify-between">
            {/* アプリタイトル (左側) */}
            <div>
              {/* sm:text-3xl は「sm (640px) 以上の画面では text-3xl」というレスポンス指定。
                狭い画面では text-2xl で控えめに、広い画面では大きく表示する。 */}
              <h1 className="text-2xl sm:text-3xl font-bold text-gray-900">
                書籍管理アプリ
              </h1>
              <p className="mt-1 text-sm text-gray-500">
                あなたの読書記録を管理しましょう
              </p>
            </div>

            {/* 新規登録ボタン (右側)。
              <Link> は内部的に <a> をレンダリングするが、JavaScript で画面遷移を高速化する。
              href="/books/new" は app/books/new/page.tsx に対応。 */}
            <Link
              href="/books/new"
              className="
                inline-flex items-center gap-2 /* インライン・フレックス: SVGとテキストを
                横並び・中央揃え */
                bg-blue-600
                text-white
                px-4 py-2.5
                rounded-lg
                font-medium
                text-sm

```

```

        hover:bg-blue-700
        transition-colors
        duration-200
        shadow-sm
    "
>
    { /* + アイコン。SVG (Scalable Vector Graphics) で「拡大しても綺麗な画像」を描
<。

    w-5 h-5 = 幅・高さともに 20px。
    fill="none" stroke="currentColor" は「中身は塗らず、線の色は親のテキスト色
    を引き継ぐ」。

    viewBox="0 0 24 24" は内部座標系 (左上(0,0)～右下(24,24))。
    <path d="M12 4v16m8-8H4" />
        M12 4 → x=12,y=4 に移動 v16 → 縦に16進む (縦線)
        m8 -8 → 相対移動 H4 → 水平に x=4 まで進む (横線)
        結果として「+」マークが描かれる。 */}

    <svg
        className="w-5 h-5"
        fill="none"
        stroke="currentColor"
        viewBox="0 0 24 24"
    >
        <path
            strokeLinecap="round" // 線の端を丸くする
            strokeLinejoin="round" // 線の交点も丸くする
            strokeWidth={2} // 線の太さ
            d="M12 4v16m8-8H4"
        />
    </svg>
    新規登録
</Link>
</div>
</div>
</header>

{ /* ===== メインコンテンツ ===== */}
{ /* <main> はページの主要コンテンツを表すセマンティック要素。 */}
<main className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
    { /* 書籍数の表示 */}
    <div className="mb-6">
        <p className="text-sm text-gray-600">
            { /* {bookList.length} で配列の要素数 (書籍冊数) を JSX に埋め込む。
                JSX では中括弧 {} の中に JavaScript 式を書ける。 */}
            全 <span className="font-bold text-gray-900">{bookList.length}</span> 冊
        </p>
    </div>

    { /* 書籍一覧 - 子コンポーネント BookList に books 配列を Props として渡す。
        BookList は中で 0冊なら空状態、1冊以上ならカードグリッドを表示する。 */}
    <BookList books={bookList} />

```

```

    </main>
  </div>
);
}

```

画面全体はこのように表示されます:

画面は大きく **ヘッダー** と **メインコンテンツ** の2つの領域に分かれています。背景は薄いグレー（ `bg-gray-50` ）です。

ヘッダー部分: - 白い背景（ `bg-white` ）に薄い影（ `shadow-sm` ）と下ボーダーがついた帯状のヘッダーです。 - 左側に「書籍管理アプリ」というアプリタイトルが大きな太字で表示され、その下に「あなたの読書記録を管理しましょう」という小さな説明文があります。 - 右側に青いボタン「+ 新規登録」が配置されています。ボタンの左にはプラスアイコン（SVG）が表示されます。 - ヘッダーの内容は `max-w-7xl`（1280px）で中央揃えされ、画面幅が広くても広がりにすぎません。

メインコンテンツ部分: - ヘッダーの下に「全 N 冊」という書籍数のテキストがあります。数字部分は太字です。 - その下に BookList コンポーネントが配置されます。 - **書籍が登録されている場合:** 白いカードがグリッド状に並びます。スマホでは1列、タブレットでは2列、デスクトップでは3列です。各カードの間には適度な余白があります。 - **書籍が0冊の場合:** 画面中央に本の絵文字と「書籍が登録されていません」というメッセージ、そして「最初の書籍を登録する」ボタンが表示されます。

例として、3冊の書籍が登録されている場合のデスクトップ表示をイメージすると:



1f. データ取得フローの図解

以下は、トップページが表示されるまでのデータの流れを示す Mermaid シーケンス図です。



ポイント:

- Server Component はサーバー側で実行されるため、Supabase へのリクエストもサーバーから行われます。クライアント (ブラウザ) から直接データベースにアクセスすることはありません。
- ブラウザは完成した HTML を受け取って表示するだけなので、初回表示が高速です。
- Supabase のシークレットキーがブラウザに漏れることもありません (サーバー側でのみ使用されるため)。

2. ローディング状態

Next.js App Router では、`loading.tsx` ファイルを作成すると、ページの読み込み中に自動的にローディング表示を行います。

LoadingSpinner コンポーネント

ファイル: `components/LoadingSpinner.tsx`

▼ このコードがやること（先に日本語で）：データ読み込み中に表示する、くるくる回るスピナー（読み込み中マーク）の部品を作ります。仕組みはシンプルで、円のうち1辺だけ色を変え、それを CSS アニメーション（`animate-spin`）で回し続けて「処理中」を表現します。サイズ（小・中・大）やスピナー下のメッセージは Props で切り替えられるようにします。初心者は「回転する円と説明文を出すだけの部品」と捉えればOKです（サイズの辞書パターンなどはコード内コメントにあります）。

```
// components/LoadingSpinner.tsx
"use client"; // このファイル内のコンポーネントは Client Component として扱う宣言。必ず1行目に書く。

/**
 * LoadingSpinner - ローディング中に表示するスピナーコンポーネント
 *
 * CSS アニメーション (animate-spin) で回転する円を表示する。
 * "use client" を付けているのは、アニメーションがクライアント側で
 * 実行されるため (厳密には Server Component でも動作するが、
 * 再利用性のために Client Component にしている)。
 */

type LoadingSpinnerProps = {
  /** スピナーのサイズ (デフォルト: "md")。?: は「省略可能」の意味。 */
  size?: "sm" | "md" | "lg"; // ユニオン型 = この3つの文字列リテラルのうちどれか
  /** スピナーの下に表示するテキスト。省略可。 */
  message?: string;
};

// 引数で size と message にデフォルト値を設定 (呼び出し側で省略された時に使う値)。
export default function LoadingSpinner({
  size = "md",
  message = "読み込み中...",
}: LoadingSpinnerProps) {
  // サイズに応じたクラスを定義 (StatusBadge と同じ「辞書パターン」)。
  // キー: サイズ識別子、値: Tailwind の幅・高さ・枠線太さのクラス文字列。
  const sizeClasses = {
    sm: "w-6 h-6 border-2", // 24px × 24px、枠線2px
    md: "w-10 h-10 border-3", // 40px × 40px、枠線3px
    lg: "w-16 h-16 border-4", // 64px × 64px、枠線4px
  };

  return (
    // flex-col = 縦並び (column)、items-center / justify-center = 縦横とも中央揃え。
    <div className="flex flex-col items-center justify-center py-16">
      {/* 回転するスピナー: 円形のうち1辺だけ色を変えて、それを回転させることで「読み込み中」を表現
      する。 */}
      <div
        className={`
          ${sizeClasses[size]}          /* 上で選んだサイズクラスを展開 (テンプレートリテラル) */
          border-gray-300                /* 枠線の基本色: 薄いグレー */
          border-t-blue-600              /* 上辺だけ青に: これが回ると進捗感が出る */
          rounded-full                   /* 完全な円形に */
          animate-spin                  /* Tailwind のクラスで「回り続ける」アニメーションを適用
        */
      `}
        role="status"                  // スクリーンリーダーに「ステータス表示」と伝える
        aria-label="読み込み中"        // 読み上げ文言
      </div>
    </div>
  );
}
```

```

    />
    { /* メッセージが渡されていれば下に表示。
        「&&」は短絡評価：左辺が真なら右辺の JSX を描画、偽（空文字や undefined）なら何も描画し
        ない。 */
        {message && (
            <p className="mt-4 text-sm text-gray-500">
                {message}
            </p>
        )}
    }
</div>
);
}

```

loading.tsx

ファイル: `app/loading.tsx`

▼ このコードがやること（先に日本語で）：ページの読み込み中に自動で表示される「待ち画面」を作ります。Next.js には「`loading.tsx` という名前のファイルを置くと、同じ場所の `page.tsx` がデータを取得している間、自動でこの画面を出してくれる」という決まりがあり、それを使います。中身は、ヘッダーの形だけ先に出すグレーの矩形（スケルトンUI）と、さきほど作った `LoadingSpinner` です。初心者は「ファイル名を `loading.tsx` にするだけで、読み込み中に勝手に出る画面になる」という1点を押さえれば十分です（詳細はコード内コメントにあります）。

```
// app/loading.tsx ← Next.js のルールで「loading.tsx」というファイル名にすると、
// 同じ階層の page.tsx の読み込み中に自動でこの画面が表示される。

import LoadingSpinner from "@/components/LoadingSpinner";

/**
 * ルートレイアウトのローディング UI
 *
 * Next.js App Router は、ページコンポーネント (page.tsx) の
 * データ取得中にこの loading.tsx を自動的に表示する。
 *
 * 仕組み:
 * 1. ユーザーがページに遷移する
 * 2. page.tsx 内の async 処理 (Supabase からのデータ取得等) が実行される
 * 3. その間、この loading.tsx が表示される
 * 4. データ取得が完了すると、page.tsx の内容に自動的に切り替わる
 *
 * これは React の Suspense (サスペンス: 読み込み中のフォールバック表示機能) を内部的に利用している。
 */
export default function Loading() { // async は不要: データ取得しない静的な画面
  return (
    <div className="min-h-screen bg-gray-50">
      {/* ヘッダーのスケルトン (ローディング中もヘッダーのような見た目を維持)。
       スケルトン UI (skeleton UI) とは「コンテンツの形だけ先に出す」UI手法。
       完成形と同じ位置にグレーの矩形を置くことで、レイアウトの「ガタつき」を防げる。 */}
      <header className="bg-white shadow-sm border-b border-gray-200">
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-6">
          <div className="flex items-center justify-between">
            <div>
              {/* h-8 = 高さ32px、w-48 = 幅192px。タイトル位置にグレーの矩形を置く。
               animate-pulse = ゆっくり点滅するアニメーション (読み込み感を演出)。 */}
              <div className="h-8 w-48 bg-gray-200 rounded animate-pulse" />
              {/* mt-2 = 上方向のマージン8px。説明文の位置に小さい矩形。 */}
              <div className="mt-2 h-4 w-64 bg-gray-100 rounded animate-pulse" />
            </div>
            <div>
              {/* ボタン位置の矩形。実際のボタンとほぼ同じ大きさ。 */}
              <div className="h-10 w-28 bg-gray-200 rounded-lg animate-pulse" />
            </div>
          </div>
        </div>
      </header>

      {/* メインコンテンツのローディング表示。
       LoadingSpinner に size="lg" (大きいサイズ) と message を渡す。 */}
      <main className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
        <LoadingSpinner size="lg" message="書籍データを読み込んでいます..." />
      </main>
    </div>
  )
}
```

```
);  
}
```

表示の説明:

ローディング中は、ヘッダー部分がスケルトン UI（灰色の長方形がパルスアニメーションする）として表示され、メインコンテンツ領域の中央に大きなスピナー（回転する円）と「書籍データを読み込んでいます...」というメッセージが表示されます。ユーザーは「何かが読み込まれている」ことをすぐに理解できます。

3. エラーハンドリング

ファイル: `app/error.tsx`

ページのレンダリング中にエラーが発生した場合に表示される画面です。Next.js App Router の Error Boundary 機能を利用しています。

▼ このコードがやること（先に日本語で）：ページの処理中にエラーが起きたとき、自動で表示される「エラー画面」を作ります。Next.js では「`error.tsx`」という名前のファイルを置くと、そのページで想定外のエラーが起きたときに自動で出してくれる決まりがあり、それを使います。画面には警告アイコンとお詫びメッセージ、そしてもう一度試すボタン（`reset` 関数を呼ぶ）を置きます。注意点は、この `error.tsx` は必ず先頭に `"use client"` を書く必要があること（Next.js の仕様）です。初心者は「エラー時の専用画面で、再試行ボタンを出すもの」と押さえれば十分です（詳細はコード内コメントにあります）。

```
// app/error.tsx ← Next.js の規約で、ページレンダリング中に未処理エラーが起きた時に自動表示される。
"use client"; // 必須: Next.js の仕様で error.tsx は Client Component でなければならない

/**
 * エラーページ
 *
 * 重要: error.tsx は必ず "use client" でなければならない。
 * これは Next.js の仕様で、Error Boundary（エラー境界：内部のエラーを捕まえる仕組み）は
 * Client Component である必要がある。
 *
 * このコンポーネントは以下の場合に自動的に表示される:
 * - Server Component でエラーが throw された場合
 * - Client Component で未処理のエラーが発生した場合
 * - Supabase からのデータ取得が失敗した場合 (throw した場合)
 */

type ErrorPageProps = {
  /** 発生したエラーオブジェクト。Error 型に digest (Next.js が付与するエラーID) を追加した型。 */
  error: Error & { digest?: string }; // 「&」は交差型：両方の性質を併せ持つ
  /** エラーからの復帰を試みる関数（ページの再レンダリングを試行する）。
   * 「() => void」は「引数なし、戻り値なし」の関数型。 */
  reset: () => void;
};

export default function ErrorPage({ error, reset }: ErrorPageProps) {
  return (
    // 画面全体を埋め、その中央に1枚のカードを配置するレイアウト。
    // flex items-center justify-center で「子要素を縦横中央寄せ」を実現。
    <div className="min-h-screen bg-gray-50 flex items-center justify-center px-4">
      { /* max-w-md = 最大幅 28rem (約448px)、w-full = 親いっぱい。
       * 結果として「最大448pxまで広がり、それ以上は広がらない」カードになる。 */ }
      <div className="max-w-md w-full bg-white rounded-lg shadow-lg p-8 text-center">
        { /* エラーアイコン。赤い円の中に三角形の警告マーク。
         * mx-auto = 左右マージン自動 (中央寄せ)。 */ }
        <div className="mx-auto w-16 h-16 bg-red-100 rounded-full flex items-center justify-center mb-6">
          <svg
            className="w-8 h-8 text-red-600"
            fill="none"
            stroke="currentColor"
            viewBox="0 0 24 24"
          >
            { /* この path 文字列は警告三角形 (中に「!」のような縦線と点) を描画している。 */ }
            <path
              strokeLinecap="round"
              strokeLinejoin="round"
              strokeWidth={2}

```

```

        d="M12 9v2m0 4h.01m-6.938 4h13.856c1.54 0 2.502-1.667 1.732-2.5L13.732
4c-.77-.833-1.964-.833-2.732 0L4.082 16.5c-.77.833.192 2.5 1.732 2.5z"
    />
</svg>
</div>

{/* エラーメッセージ */}
<h2 className="text-xl font-bold text-gray-900 mb-2">
    エラーが発生しました
</h2>
<p className="text-gray-600 mb-6">
    申し訳ありません。データの取得中にエラーが発生しました。
    しばらく時間をおいてから再度お試しください。
</p>

{/* エラー詳細（開発時のデバッグ用）。
    process.env.NODE_ENV は Node.js が提供する環境変数。
        "development" → 開発中 (npm run dev)
        "production" → 本番ビルド (npm run build → npm start)
    開発時だけ詳細表示することで、本番ユーザーに技術的な情報を見せないようにする。 */}
{process.env.NODE_ENV === "development" && (
    // <details> / <summary> は HTML 標準の「折りたたみ」要素。
    // クリックで <summary> 以外の中身が開く。JS 不要。
    <details className="mb-6 text-left">
        <summary className="text-sm text-gray-500 cursor-pointer hover:text-gray-
700">
            エラー詳細を表示
        </summary>
        {/* <pre> はそのままの体裁（改行・空白）を保持して表示する HTML 要素。
            overflow-auto max-h-40 で「内容が多ければスクロール、最大160pxまで」。 */}
        <pre className="mt-2 p-3 bg-gray-100 rounded text-xs text-red-600 overflow-
auto max-h-40">
            {error.message}    {/* Error オブジェクトの message プロパティ（人間可読なエラー
説明文）。 */}
        </pre>
    </details>
)}

{/* アクションボタン群。
    flex-col sm:flex-row で「スマホでは縦並び、smサイズ以上では横並び」を切り替え。 */}
<div className="flex flex-col sm:flex-row gap-3 justify-center">
    {/* reset 関数を onClick に渡す。Next.js が用意するこの関数を呼ぶと、
        ページの再レンダリングが試みられ、エラーから復帰できる可能性がある。 */}
    <button
        onClick={reset}
        className="
            px-6 py-2.5
            bg-blue-600
            text-white
            rounded-lg

```



```

        font-medium
        hover:bg-blue-700
        transition-colors
        duration-200
    "
    >
        もう一度試す
    </button>
    {/* トップページへ戻るリンク。エラー時は <Link> ではなく普通の <a> を使うことで
        「アプリ全体を完全に初期化」できる（クライアントサイドキャッシュも含めてリセット）。
    */}

    <a
        href="/"
        className="
            px-6 py-2.5
            bg-gray-100
            text-gray-700
            rounded-lg
            font-medium
            hover:bg-gray-200
            transition-colors
            duration-200
        "
    >
        トップに戻る
    </a>
</div>
</div>
</div>
);
}

```

表示の説明:

画面中央に白いカードが表示されます。赤い三角形の警告アイコン、「エラーが発生しました」というメッセージ、そして「もう一度試す」ボタンと「トップに戻る」ボタンが配置されます。開発環境（`NODE_ENV === "development"`）では、エラーの詳細メッセージも折りたたみで確認できます。

4. 登録機能の実装

ここからは書籍の新規登録機能を作ります。ユーザーがフォームに情報を入力し、Supabase のデータベースに新しいレコードを INSERT する流れです。

4-0. 登録機能を作る前に知っておきたい用語

実装に入る前に、いくつかの言葉の意味を押さえておきましょう。

用語	読み・英語	1行説明
フォーム	form	ユーザーが入力する箱（input、textarea、selectなど）を集めた HTML 要素 <code><form></code>
FormData	フォームデータ	フォームの入力内容をまとめて持つブラウザ標準のオブジェクト
Server Action	サーバーアクション	Next.js の機能。Client側から呼べるけど実行はサーバーで行われる関数
revalidatePath	リバリデートパス	指定したパスのキャッシュを破棄して次回アクセス時に作り直す関数
redirect	リダイレクト	別のURLにブラウザを移動させる関数
INSERT	インサート	SQL で「新しい行を追加する」操作

本書での登録機能のアプローチ: Next.js には「Server Action」というサーバー側で実行される関数を使う登録方式と、「ブラウザ側の Supabase クライアントから直接 INSERT」する方式の2通りがあります。本書ではこの章では後者（クライアント方式）を採用します。理由は、Reactの useState や onSubmit といった基本機能だけで完結し、初学者にとって流れが追いやすいためです。

Server Action 方式は仕組みがやや高度（ 'use server' ディレクティブ、`<form action={fn}>`、FormData の扱い、revalidatePath、redirect などの理解が必要）なので、参考として下の「Server Action 方式の概要」コラムにまとめておきます。実装は読み飛ばしても構いません。

コラム: Server Action 方式の概要（参考）

参考までに、Server Action を使うとどんなコードになるかだけ簡単に紹介します。この章では実装しません（クライアント方式で進めます）。

```
// app/actions.ts - Server Action を定義するファイル例 (参考)
"use server"; // ファイル先頭でこのディレクティブを書くと、中の関数はすべてサーバー実行になる

import { createClient } from "@lib/supabase/server"; // サーバー用 Supabase クライアント
import { revalidatePath } from "next/cache"; // 指定パスのキャッシュを破棄する関数
import { redirect } from "next/navigation"; // 別のページに遷移させる関数

// フォーム送信時に呼ばれる Server Action。引数 formData はブラウザ標準の FormData オブジェクト。
// FormData とは <form> 内の入力値 (name 属性をキーに) をまとめた箱。
export async function createBook(formData: FormData) {
  // formData.get("title") の戻り値の型は FormDataEntryValue | null。
  // FormDataEntryValue は「string か File のどちらか」。
  // 通常のテキスト入力なら string になるが、TypeScript 上はそうとは限らないので、
  // String(...) で文字列にキャストするのが安全 (null なら "null" になるので注意)。
  const title = String(formData.get("title") ?? "");
  const author = String(formData.get("author") ?? "");

  const supabase = await createClient();
  await supabase.from("books").insert({ title, author });

  // 関連パスのキャッシュを破棄 → 次の表示で最新データが取られる
  revalidatePath("/");

  // 登録完了後にトップへ遷移
  redirect("/");
}
```

```
// app/books/new/page.tsx - Server Action を使う側 (参考)
import { createBook } from "@app/actions";

export default function NewBookPage() {
  return (
    // <form action={createBook}> と書くと、送信時に Next.js が自動で
    // FormData を組み立てて createBook(formData) を呼んでくれる。
    <form action={createBook}>
      <input name="title" /> { /* name="title" が formData.get("title") に対応 */ }
      <input name="author" />
      <button type="submit">登録</button>
    </form>
  );
}
```

Server Action のメリット (参考) : - ブラウザ側 JS の量が減る (送信ロジックがサーバーに置かれる)。 - Supabase の秘密鍵などをサーバー側だけで使える。 - フォーム送信時に JavaScript が無効でも動作する (プログレッシブエンハンスメ

ント）。

Server Action のセキュリティ: - Server Action はサーバー上で実行されるため、フォームに「価格を勝手に書き換える」「他人の ID を送る」といった改ざんが届いても、Server Action の中で再検証すれば防げます。ただし「クライアントから送られた値は信用しない」前提でバリデーションを書くことが必須です。

Server Action と組み合わせるフック（参考）: - `useFormState` / `useActionState`（React 19以降）: フォーム送信の結果（成功/エラー）を state として保持する。 - `useFormStatus`: フォーム送信中かどうか（pending状態）を取得する。

これらは第8章以降のオプションとして紹介します。

4a. BookForm コンポーネント

ファイル: `components/BookForm.tsx`

フォーム全体を管理するコンポーネントです。新規登録と編集の両方で使えるように設計します（この章では新規登録のみ使用、編集は次章で使用）。

▼ このコードがやること（先に日本語で）: 書籍情報を入力するフォーム本体を作ります。カギは3つ——①入力中の値を `useState` という「変化する箱」に1つのオブジェクトとしてまとめて持つこと、②入力が変わるたびにその箱を更新する（制御コンポーネント＝画面の値とプログラムの値を常に一致させる方式）こと、③送信前に「タイトルは必須」などの入力チェック（バリデーション）を行い、問題があればエラーを表示して送信を止めること。実際のデータベース保存は、このフォームではなく親から渡された `onSubmit` 関数に任せる設計です（登録と編集で同じフォームを使い回すため）。初心者は「入力を集めてチェックし、OKなら親に渡す係」と押さえれば十分です（詳細はコード内コメントにあります）。

```
// components/BookForm.tsx
"use client"; // useState などブラウザ専用機能を使うので Client Component 化

/**
 * BookForm - 書籍の登録・編集フォームコンポーネント
 *
 * "use client" が必要な理由:
 * - useState でフォームの入力状態を管理する
 * - onChange イベントハンドラを使う
 * - フォーム送信時の処理 (onSubmit) をハンドリングする
 * これらはすべてブラウザ側で動作する機能のため、Client Component にする。
 *
 * このコンポーネントは「登録」と「編集」の両方で使い回す。
 * - 新規登録時: initialData を渡さない → 空のフォーム
 * - 編集時: initialData に既存データを渡す → 値が入ったフォーム
 */

// useState は「状態」を管理するための React フック。
// type FormEvent は「フォーム送信イベントオブジェクト」の型。"type" を付けると型だけインポート。
import { useState, type FormEvent } from "react";
// 同フォルダの StatusBadge から BookStatus 型だけインポート (値は使わない)
import { type BookStatus } from "./StatusBadge";

// フォームのデータ型 (データベースの books テーブルに対応するが、
// id, created_at, updated_at はフォームでは扱わない)。
// export しているので、親ページがこの型を使って handleSubmit の引数を型付けできる。
export type BookFormData = {
  title: string; // タイトル (必須)
  author: string; // 著者 (必須)
  publisher: string; // 出版社 (任意)。文字列のまま扱い、null は親側で変換する
  published_date: string; // 出版日 (任意)。“YYYY-MM-DD” 形式の文字列
  rating: number | null; // 評価 (任意)。1~5の数値 または null
  status: BookStatus; // ステータス (必須)。“reading” | “completed” |
  “want_to_read”
  memo: string; // メモ (任意)
};

// Props の型定義 (親から受け取る値の形)
type BookFormProps = {
  /** 編集時に渡す初期データ (新規登録時は undefined)。?: で省略可能。 */
  initialData?: BookFormData;
  /** フォーム送信時に呼ばれるコールバック関数。
   * - 引数: フォームのデータ
   * - 戻り値: Promise<void> (非同期処理を行う想定。中で await できる)
   */
  onSubmit: (data: BookFormData) => Promise<void>;
  /** 送信ボタンのテキスト (“登録する” / “更新する” など)。省略時はデフォルト値を使う。 */
  submitLabel?: string;
  /** 送信中かどうか (ボタンの無効化・ローディング表示に使う) */

```

```

    isSubmitting?: boolean;
};

// フォームの初期値（新規登録時に使う）。すべての項目を「空」または「初期選択」にしておく。
const defaultFormData: BookFormData = {
  title: "",
  author: "",
  publisher: "",
  published_date: "",
  rating: null, // 「未選択」を表す null
  status: "want_to_read", // 新規登録時は「読みたい」を初期値に
  memo: "",
};

export default function BookForm({
  initialData,
  onSubmit,
  submitLabel = "登録する", // デフォルト値：呼び出し側が省略したらこれを使う
  isSubmitting = false,
}: BookFormProps) {
  // ----- State -----
  // フォームの入力値を管理する state。
  // useState<型>([初期値]) で「型と初期値」を指定する。
  // initialData が渡されていればそれを使い、なければデフォルト値を使う（?? は nullish
  coalescing）。
  const [formData, setFormData] = useState<BookFormData>([
    initialData ?? defaultFormData
  ]);

  // バリデーションエラーを管理する state。
  // Partial<T> = T の全プロパティを「省略可能」にした型
  // Record<K, V> = キーが K、値が V のオブジェクト型
  // keyof BookFormData = BookFormData のプロパティ名のユニオン型 ("title" | "author" |
  ...)
  // つまり {title?: string; author?: string; ...} という型。
  const [errors, setErrors] = useState<Partial<Record<keyof BookFormData, string>>>
    ({});

  // ----- バリデーション -----
  /**
   * フォーム全体のバリデーションを行う
   * @returns バリデーションを通過したら true、エラーがあれば false
   */
  const validate = (): boolean => { // 戻り値型
    // 明示
    const newErrors: Partial<Record<keyof BookFormData, string>> = {}; // 新しいエ
    ラー集を作る

    // タイトル: 必須、100文字以内
    // .trim() は前後の空白文字を削った文字列を返す。

```

```

// 空文字列 "" は falsy なので、! を付けると「空または空白だけ」を検出できる。
if (!formData.title.trim()) {
  newErrors.title = "タイトルは必須です";
} else if (formData.title.trim().length > 100) {
  newErrors.title = "タイトルは100文字以内で入力してください";
}

// 著者: 必須、50文字以内
if (!formData.author.trim()) {
  newErrors.author = "著者は必須です";
} else if (formData.author.trim().length > 50) {
  newErrors.author = "著者は50文字以内で入力してください";
}

// 出版社: 任意、50文字以内 (空ならエラーにしない)
if (formData.publisher.trim().length > 50) {
  newErrors.publisher = "出版社は50文字以内で入力してください";
}

// メモ: 任意、1000文字以内
if (formData.memo.trim().length > 1000) {
  newErrors.memo = "メモは1000文字以内で入力してください";
}

// 集めたエラーを state に反映 → 画面に表示される
setErrors(newErrors);

// Object.keys(obj) は obj のキー名を配列で返す。
// エラーが1つもなければ length が 0 になり、true を返す。
return Object.keys(newErrors).length === 0;
};

// ----- イベントハンドラ -----

/**
 * テキスト入力フィールドの変更ハンドラ
 * input, textarea, select の onChange に共通で使える
 *
 * -----
 * (a) 引数 e は「変更イベントオブジェクト」。ユーザーが文字を打つと
 *     React がこのオブジェクトをハンドラに渡してくる。
 * (b) e.target は「変更が起きた DOM 要素」。ここから name と value を取り出す。
 *     - name : <input name="title"> のように JSX で指定した名前
 *     - value : 現在の入力値 (文字列)
 * -----
 */
const handleChange = (
  e: React.ChangeEvent<HTMLInputElement | HTMLTextAreaElement | HTMLSelectElement>
) => {
  // (1) e.target から name, value を分割代入で取り出す

```

```

// 例: <input name="title" value="..." /> なら
//      name = "title", value = ユーザーが打った文字列
const { name, value } = e.target;

// (2) 現在の formData を「コピーして1つの項目だけ書き換える」更新パターン
//      setData((prev) => ...) の prev は「直前の formData の値」。
//      ...prev は「prev の中身を全部コピー」(スプレッド構文)。
//      [name]: value は「name 変数の値をキーにして value を入れる」記法
//      (Computed Property Names)。
//
// 例: name="title", value="React入門" のとき
//      { ...prev, title: "React入門" } という新しいオブジェクトを作る。
//      React は「formData が新しいオブジェクトに入れ替わった」と検知して
//      再描画する。直接 prev.title = value のように書き換えてはいけない。
setData((prev) => ({
  ...prev,
  [name]: value,
}));

// (3) ユーザーが入力をやり直し始めたら、そのフィールドの古いエラーを消す。
//      [name]: undefined を入れることでエラー表示が消える。
//      `as keyof BookFormData` は「name は BookFormData のキーですよ」と
//      TS に教えるアサーション(型注釈の補助)。
if (errors[name as keyof BookFormData]) {
  setErrors((prev) => ({
    ...prev,
    [name]: undefined,
  }));
}
};

/**
 * 評価 (rating) の変更ハンドラ
 * select の値は文字列なので、数値に変換する必要がある。
 * 「選択してください」を選んだ時は value が空文字列 "" になるので、それを null に変換する。
 */
const handleRatingChange = (e: React.ChangeEvent<HTMLSelectElement>) => {
  // HTMLSelectElement の value は常に string。<option value="3"> なら "3"。
  const value = e.target.value;
  setData((prev) => ({
    ...prev,
    // 三項演算子: value === "" なら null、それ以外なら Number(value) で数値化。
    // Number("3") は 3 を返す。Number("abc") は NaN を返すので、数値以外が来る場合は注意が必要。
    // ここでは <option value="..."> の値は固定なので問題ない。
    rating: value === "" ? null : Number(value),
  }));
};

/**

```



```

* フォーム送信ハンドラ
* <form onSubmit={handleSubmit}> から呼ばれる。
* - 引数 e はフォーム送信イベントオブジェクト。
* - 関数全体に async が付くので、内部で await が使える。
*/
const handleSubmit = async (e: FormEvent) => {
  // (1) e.preventDefault() を呼ばないと、ブラウザが既定の動作として
  //     ページ全体をリロードしてしまう (HTML本来の挙動)。
  //     SPA では絶対に必要な「お決まりの一行」。
  e.preventDefault();

  // (2) バリデーション関数を呼び、エラーがあれば中断
  //     validate() は内部で setErrors を呼ぶので、画面にエラー文も表示される。
  if (!validate()) {
    return; // 関数を終了 (送信しない)
  }

  // (3) 親コンポーネントから渡された onSubmit コールバックを呼び
  //     onSubmit は Promise を返す async 関数なので await で待つ。
  //     ここで実際のDB書き込みやページ遷移が行われる (実装は親側にある)。
  await onSubmit(formData);
};

// ----- 表示 -----
return (
  // <form> は HTML 標準のフォーム要素。
  // onSubmit={handleSubmit} で送信時に handleSubmit を呼び。
  // space-y-6 は Tailwind の「直下の子要素の間隔を縦方向に 6 (24px) 空ける」ユーティリティ。
  <form onSubmit={handleSubmit} className="space-y-6">
    {/* ===== タイトル ===== */}
    <div>
      {/* <label htmlFor="title"> は「title という id を持つ入力要素のラベル」という関連付
      け。
      ラベルをクリックすると、関連付けられた入力にフォーカスに移る。
      ※React では「for」が JS の予約語と被るので、HTML の for 属性は「htmlFor」と書く。
      */}
      <label
        htmlFor="title"
        className="block text-sm font-medium text-gray-700 mb-1"
      >
        タイトル <span className="text-red-500">*</span> {/* 赤い「*」で必須を表す */}
      </label>
      <input
        type="text" // 1行のテキスト入力
        id="title" // label の htmlFor と一致させる
        name="title" // FormData で使うキー名 (Server Action方式の
        // 場合に重要)
        value={formData.title} // 制御コンポーネント: state の値を入力欄の値とし
        // て使う
        onChange={handleChange} // 入力が変わったら handleChange を呼び

```

```

placeholder="例: リーダブルコード"
// テンプレートリテラル内で「エラーがあれば赤い枠線、なければ通常の枠線」を切り替える。
className={`
  w-full px-4 py-2.5
  border rounded-lg
  text-gray-900
  placeholder-gray-400
  focus:outline-none focus:ring-2 focus:ring-blue-500 focus:border-
transparent /* フォーカス時に青い縁取り */
  transition-colors duration-200
  ${errors.title ? "border-red-500 bg-red-50" : "border-gray-300"}
`}
/>
{/* エラーがあれば、フィールドのすぐ下に赤字でエラーメッセージを表示。
  「errors.title &&」は短絡評価：左辺が真（文字列がある）なら右辺の JSX を描画。 */}
{errors.title && (
  <p className="mt-1 text-sm text-red-600">{errors.title}</p>
)}
</div>

{/* ===== 著者 ===== */}
<div>
  <label
    htmlFor="author"
    className="block text-sm font-medium text-gray-700 mb-1"
  >
    著者 <span className="text-red-500">*</span>
  </label>
  <input
    type="text"
    id="author"
    name="author"
    value={formData.author}
    onChange={handleChange}
    placeholder="例: Dustin Boswell"
    className={`
      w-full px-4 py-2.5
      border rounded-lg
      text-gray-900
      placeholder-gray-400
      focus:outline-none focus:ring-2 focus:ring-blue-500 focus:border-
transparent
      transition-colors duration-200
      ${errors.author ? "border-red-500 bg-red-50" : "border-gray-300"}
    `}
  />
  {errors.author && (
    <p className="mt-1 text-sm text-red-600">{errors.author}</p>
  )}
</div>

```

```

{ /* ===== 出版社 ===== */}
<div>
  <label
    htmlFor="publisher"
    className="block text-sm font-medium text-gray-700 mb-1"
  >
    出版社
  </label>
  <input
    type="text"
    id="publisher"
    name="publisher"
    value={formData.publisher}
    onChange={handleChange}
    placeholder="例: オライリージャパン"
    className={`
      w-full px-4 py-2.5
      border rounded-lg
      text-gray-900
      placeholder-gray-400
      focus:outline-none focus:ring-2 focus:ring-blue-500 focus:border-
transparent
      transition-colors duration-200
      ${errors.publisher ? "border-red-500 bg-red-50" : "border-gray-300"}
    `}
  />
  {errors.publisher && (
    <p className="mt-1 text-sm text-red-600">{errors.publisher}</p>
  )}
</div>

{ /* ===== 出版日 ===== */}
<div>
  <label
    htmlFor="published_date"
    className="block text-sm font-medium text-gray-700 mb-1"
  >
    出版日
  </label>
  { /* type="date" にすると、ブラウザがカレンダー UI を出してくれる。
    値は "YYYY-MM-DD" 形式の文字列で取得できる (ISO 8601 形式の一部) 。 */}
  <input
    type="date"
    id="published_date"
    name="published_date"
    value={formData.published_date}
    onChange={handleChange}
    className="
      w-full px-4 py-2.5

```

```

        border border-gray-300 rounded-lg
        text-gray-900
        focus:outline-none focus:ring-2 focus:ring-blue-500 focus:border-
transparent
        transition-colors duration-200
    "
    />
</div>

{ /* ===== 評価 と ステータス を横に並べる =====
    grid-cols-1 sm:grid-cols-2 で「スマホでは1列、smサイズ以上では2列」のグリッドに。
*/ }

<div className="grid grid-cols-1 sm:grid-cols-2 gap-6">
    { /* 評価 */ }
    <div>
        <label
            htmlFor="rating"
            className="block text-sm font-medium text-gray-700 mb-1"
        >
            評価
        </label>
        { /* <select> はドロップダウン。
            value={formData.rating ?? ""} で「null なら空文字列」を value にする。
            これは <option value=""> と一致するので「選択してください」が選ばれた状態になる。
        */ }

        <select
            id="rating"
            name="rating"
            value={formData.rating ?? ""}
            onChange={handleRatingChange} // rating は数値変換が必要なので別ハンドラ
            className="
                w-full px-4 py-2.5
                border border-gray-300 rounded-lg
                text-gray-900
                focus:outline-none focus:ring-2 focus:ring-blue-500 focus:border-
transparent
                transition-colors duration-200
                bg-white
            "
        >
            { /* <option> の value 属性が、選択時に select の value になる値。 */ }
            <option value="">選択してください</option>
            <option value="1">★☆☆☆☆ (1)</option>
            <option value="2">★★☆☆☆ (2)</option>
            <option value="3">★★★☆☆ (3)</option>
            <option value="4">★★★★☆ (4)</option>
            <option value="5">★★★★★ (5)</option>
        </select>
    </div>

```

```

{ /* ステータス */}
<div>
  <label
    htmlFor="status"
    className="block text-sm font-medium text-gray-700 mb-1"
  >
    ステータス <span className="text-red-500">*</span>
  </label>
  <select
    id="status"
    name="status"
    value={formData.status}
    onChange={handleChange}
    className="
      w-full px-4 py-2.5
      border border-gray-300 rounded-lg
      text-gray-900
      focus:outline-none focus:ring-2 focus:ring-blue-500 focus:border-
transparent
      transition-colors duration-200
      bg-white
    "
  >
    <option value="want_to_read">読みたい</option>
    <option value="reading">読書中</option>
    <option value="completed">読了</option>
  </select>
</div>
</div>

{ /* ===== メモ ===== */}
<div>
  <label
    htmlFor="memo"
    className="block text-sm font-medium text-gray-700 mb-1"
  >
    メモ
  </label>
  { /* <textarea> は複数行入力。input と違って終了タグが必要。
    rows={4} で「最初の表示高さ」を4行分に設定。実際には自由に広げられる (resize-
vertical)。 */}
  <textarea
    id="memo"
    name="memo"
    value={formData.memo}
    onChange={handleChange}
    rows={4}
    placeholder="この書籍に関するメモを自由に入力してください"
    className={`
      w-full px-4 py-2.5

```

```

border rounded-lg
text-gray-900
placeholder-gray-400
focus:outline-none focus:ring-2 focus:ring-blue-500 focus:border-
transparent
transition-colors duration-200
resize-vertical
/* 縦方向だけリサイズ可能 */
    ${errors.memo ? "border-red-500 bg-red-50" : "border-gray-300"}
  `
/>
{errors.memo && (
  <p className="mt-1 text-sm text-red-600">{errors.memo}</p>
)}
/* 文字数カウンター。入力するたびに再レンダリングされて数値が更新される。 */
<p className="mt-1 text-xs text-gray-400">
  {formData.memo.length} / 1000 文字
</p>
</div>

/* ===== 送信ボタン ===== */
<div className="flex items-center gap-4 pt-4">
  <button
    type="submit" // type="submit" で「これがフォームの送信ボタ
ン」と明示
    disabled={isSubmitting} // 送信中はボタンを無効化（連打防止）
    // 送信中は薄い青&クリック不可マウスカーソル、通常時は濃い青&ホバーで色変化
    className={`
      px-8 py-3
      rounded-lg
      font-medium
      text-white
      transition-colors duration-200
      ${
        isSubmitting
          ? "bg-blue-400 cursor-not-allowed"
          : "bg-blue-600 hover:bg-blue-700"
      }
    `}
  >
    /* 送信中ならスピナー + 「送信中...」、それ以外なら submitLabel ("登録する"等) を表示
    */
    {isSubmitting ? (
      <span className="flex items-center gap-2">
        /* 送信中のスピナー (SVG)。animate-spin で回転アニメーション。 */
        <svg
          className="animate-spin h-5 w-5"
          viewBox="0 0 24 24"
          fill="none"
        >

```

```

        { /* 背景の薄い円（不透明度25%）。 */ }
        <circle
            className="opacity-25"
            cx="12"                // 中心 x
            cy="12"                // 中心 y
            r="10"                 // 半径
            stroke="currentColor"
            strokeWidth="4"
        />
        { /* 上に重ねる「動く一部の弧」（不透明度75%）。これが回って見える。 */ }
        <path
            className="opacity-75"
            fill="currentColor"
            d="M4 12a8 8 0 018-8v4a4 4 0 00-4 4H4z"
        />
    </svg>
    送信中...
</span>
) : (
    submitLabel
)}
</button>

{ /* キャンセルボタン（トップページに戻る）。<a> なのでフォーム送信はしない。 */ }
<a
    href="/"
    className="
        px-6 py-3
        rounded-lg
        font-medium
        text-gray-700
        bg-gray-100
        hover:bg-gray-200
        transition-colors duration-200
    "
    >
    キャンセル
</a>
</div>
</form>
);
}

```

フォームはこのように表示されます:

白い背景のページ上に、縦に並んだフォームフィールドが表示されます。各フィールドの構成は以下の通りです。

新規書籍を登録

タイトル *

例: リーダブルコード

著者 *

例: Dustin Boswell

出版社

例: オライリージャパン

出版日

yyyy/mm/dd

評価

選択してください ▼

ステータス *

読みたい ▼

メモ

この書籍に関するメモを自由に入力してください

0 / 1000 文字

登録する

キャンセル

- 各フィールドのラベルの横にある * マーク（赤色）は必須フィールドを示します。
- 入力フィールドにフォーカスすると、青い枠線（`focus:ring-2 focus:ring-blue-500`）が表示されます。
- バリデーションエラーがある場合、そのフィールドの枠線が赤くなり（`border-red-500`）、背景が薄い赤色（`bg-red-50`）になり、フィールドの下にエラーメッセージが赤文字で表示されます。
- 「評価」と「ステータス」はデスクトップでは横に並び（`sm:grid-cols-2`）、スマホでは縦に並びます。
- メモ欄の下には入力文字数カウンター（`0 / 1000 文字`）が表示されます。
- 送信ボタンは青色で、送信中は薄い青色になりスピナーアイコンが回転します。

4b. 登録ページ

ファイル: `app/books/new/page.tsx`

BookForm を使って新規登録画面を構成するページです。

▼ このコードがやること（先に日本語で）：さきほどの `BookForm` を画面に置き、フォームが送信されたら Supabase に新しい書籍を INSERT（新規追加）し、成功したらトップページへ戻る処理を書きます。カギは送信処理の流れ——①送信中フラグを立ててボタンを無効化（二重送信防止）、② `supabase.from("books").insert({...})` でデータベースに登録、③成功なら `router.push("/")` で一覧へ戻り `router.refresh()` で最新データを取り直す、④失敗ならエラーメッセージを表示。try / catch / finally で「成功・失敗どちらでも最後にフラグを戻す」点も押さえるところです。初心者は「フォームの送信を受け取ってDBに保存し、画面を切り替える係」と捉えれば十分です（各行の意味はコード内コメントにあります）。

```
// app/books/new/page.tsx ← URL "/books/new" に対応するページ (App Router のファイルベース
ルーティング)
"use client"; // useState・useRouter などブラウザ専用機能を使うので Client Component

/**
 * 書籍登録ページ
 *
 * "use client" を付けている理由:
 * - BookForm (Client Component) の onSubmit コールバックで
 *   Supabase への INSERT 処理を行う
 * - useRouter で登録後のリダイレクトを行う
 * - useState で送信中の状態を管理する
 *
 * ※ Server Action を使う方法もあるが、この章では
 * 初心者にとって分かりやすい Client Component 方式で実装する。
 */

import { useState } from "react"; // React の状態管理フック
import { useRouter } from "next/navigation"; // Next.js App Router の
ページ遷移フック
import { createClient } from "@lib/supabase/client"; // ブラウザ用 Supabase ク
ライアント (サーバー用と別物)
import BookForm, { type BookFormData } from "@components/BookForm"; // フォーム本体と
型

export default function NewBookPage() {
  // useRouter() でルーター操作オブジェクトを取得。
  // .push("/path") で遷移、.refresh() で Server Component の再実行など。
  const router = useRouter();

  // 送信中かどうかの状態。
  // ボタンの「送信中…」表示や、二重送信防止に使う。
  const [isSubmitting, setIsSubmitting] = useState(false);

  // エラーメッセージの状態。
  // 「string | null」で「エラー文字列があるか、null (エラーなし)」を表現。
  const [submitError, setSubmitError] = useState<string | null>(null);

  /**
   * フォーム送信時の処理
   * BookForm の onSubmit に渡すコールバック関数
   */
  const handleSubmit = async (data: BookFormData) => {
    // (1) 「送信中フラグ」を ON にする → ボタンが「送信中…」表示になり無効化される
    //     setSubmitError(null) で前回のエラーメッセージをクリアする
    setIsSubmitting(true);
    setSubmitError(null);

    // (2) try / catch / finally の3ブロック構成
  }
}
```

```

//   - try      : 通常実行する処理。途中で例外 (Error) が起きると catch へ飛ぶ
//   - catch    : 想定外のエラーをキャッチ
//   - finally  : 成功・失敗どちらでも最後に必ず実行される
try {
  // (3) Supabase クライアントを作る
  //   ここではブラウザ用クライアント (ブラウザのJSから直接DBにつなぐ)。
  //   RLSポリシーがあるので、許可された操作だけが通る。
  const supabase = createClient();

  // (4) books テーブルに INSERT する
  //   .from("books") : 操作対象テーブルを指定
  //   .insert({...}) : 書き込みたいレコードのオブジェクト
  //   await          : 非同期処理を待ち、結果を { error } に展開
  //
  //   入力ガード:
  //     data.title.trim()           ← 前後の空白を削る
  //     data.publisher.trim() || null ← 空文字列なら null (DBではnullで保存)
  //     data.published_date || null  ← 同上
  //     data.memo.trim() || null     ← 同上
  //   こうすることで「空欄」と「null」を区別せずに DB に綺麗な値が入る。
  const { error } = await supabase.from("books").insert({
    title: data.title.trim(),
    author: data.author.trim(),
    publisher: data.publisher.trim() || null,
    published_date: data.published_date || null,
    rating: data.rating,
    status: data.status,
    memo: data.memo.trim() || null,
  });

  // (5) Supabase が返す error は「成功なら null、失敗ならオブジェクト」。
  //   エラーがあれば、ユーザー向けメッセージをセットして関数を終了。
  if (error) {
    console.error("書籍の登録に失敗しました:", error.message);
    setSubmitError(
      "書籍の登録に失敗しました。入力内容を確認して再度お試しください。"
    );
    return;
  }

  // (6) 登録成功 → トップページに戻る
  //   router.push("/") : URLを "/" に変える (クライアント側遷移)
  //   ページ全体のリロードは起きないので体感が速い
  router.push("/");

  // (7) router.refresh() で Server Component を再実行させ、
  //   登録したばかりのデータも含む最新の一覧を取得・表示する。
  router.refresh();
} catch (err) {
  // (8) 想定外のエラー (ネットワーク切断、JSON パース失敗など) の最後の砦

```

```

    console.error("予期しないエラー:", err);
    setSubmitError("予期しないエラーが発生しました。");
  } finally {
    // (9) 成功・失敗いずれでもボタンの「送信中」状態は解除する
    setIsSubmitting(false);
  }
};

return (
  <div className="min-h-screen bg-gray-50">
    {/* ハッダー */}
    <header className="bg-white shadow-sm border-b border-gray-200">
      {/* max-w-3xl = 最大幅 48rem (約768px)。フォームは狭いほうが読みやすいので一覧画面より狭くしている。 */}
      <div className="max-w-3xl mx-auto px-4 sm:px-6 lg:px-8 py-6">
        <div className="flex items-center gap-4">
          {/* 戻るボタン。丸い灰色ボタンに矢印アイコンを入れた「戻る」UI。
              aria-label でスクリーンリーダーに「トップに戻る」と読ませる。 */}
          <a
            href="/"
            className="
              inline-flex items-center justify-center
              w-10 h-10
              rounded-full          /* 完全に丸い形 */
              bg-gray-100
              text-gray-600
              hover:bg-gray-200
              transition-colors duration-200
            "
            aria-label="トップに戻る"
          >
            {/* 左向き矢印を描く SVG。
                d="M15 19l-7-7 7-7" は「(15,19)→(8,12)→(15,5)」と進む線（左向き「<」の
                字）。 */}
            <svg
              className="w-5 h-5"
              fill="none"
              stroke="currentColor"
              viewBox="0 0 24 24"
            >
              <path
                strokeLinecap="round"
                strokeLinejoin="round"
                strokeWidth={2}
                d="M15 19l-7-7 7-7"
              />
            </svg>
          </a>

          {/* ページタイトル */}

```

```

    <div>
      <h1 className="text-2xl font-bold text-gray-900">
        書籍を登録する
      </h1>
      <p className="mt-1 text-sm text-gray-500">
        新しい書籍の情報を入力してください
      </p>
    </div>
  </div>
</div>
</header>

{/* メインコンテンツ */}
<main className="max-w-3xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
  {/* エラーメッセージ（送信失敗時に表示）。
    submitError が null なら短絡評価で何も表示しない。
    文字列が入っているときだけ赤い警告ボックスが現れる。 */}
  {submitError && (
    <div className="mb-6 p-4 bg-red-50 border border-red-200 rounded-lg">
      <div className="flex items-center gap-2">
        {/* 円形の警告アイコン（中に「!」風の縦線と点）。
          flex-shrink-0 は「テキストが長くてもアイコンを縮めない」指定。 */}
        <svg
          className="w-5 h-5 text-red-600 flex-shrink-0"
          fill="none"
          stroke="currentColor"
          viewBox="0 0 24 24"
        >
          <path
            strokeLinecap="round"
            strokeLinejoin="round"
            strokeWidth={2}
            d="M12 8v4m0 4h.01M21 12a9 9 0 1-18 0 9 9 0 0 18 0z"
          />
        </svg>
        <p className="text-sm text-red-700">{submitError}</p>
      </div>
    </div>
  )}

  {/* フォーム本体。BookForm に必要な Props を渡す。
    - onSubmit      : 上で定義したフォーム送信処理
    - submitLabel    : ボタンに表示する文字
    - isSubmitting   : 送信中フラグ（ボタンを薄くしてスピナーを出すため） */}
  <div className="bg-white rounded-lg shadow-sm border border-gray-200 p-6 sm:p-8">
    <BookForm
      onSubmit={handleSubmit}
      submitLabel="登録する"
      isSubmitting={isSubmitting}
    />
  </div>
</main>

```

```
    />
  </div>
</main>
</div>
);
}
```

「登録ボタンを押すとうなる」の説明:

1. ユーザーがフォームの各フィールドに情報を入力し、「登録する」ボタンをクリックします。
 2. まず **クライアント側でバリデーション** が行われます。 - タイトルと著者が空の場合、赤いエラーメッセージ（「タイトルは必須です」等）が該当フィールドの下に表示され、送信は行われません。 - 文字数制限を超えている場合も同様です。
 3. バリデーションに通ると、「登録する」ボタンが **薄い青色に変わり**、スピナー（回転アイコン）と「送信中...」というテキストに変化します。ボタンは無効化（ **disabled** ）されるため、二重送信を防止します。
 4. Supabase への INSERT リクエストが **ブラウザから直接** Supabase API に送信されます。
 5. **成功した場合:** **router.push("/")** によりトップページにリダイレクトされます。一覧が再読み込みされ、今登録した書籍がリストの先頭に表示されます。画面遷移はスムーズで、ページ全体がリロードされることはありません。
 6. **失敗した場合:** フォームの上部に赤い背景のエラーメッセージ（「書籍の登録に失敗しました。入力内容を確認して再度お試しください。」）が表示されます。フォームの入力内容は保持されるので、修正して再送信できます。
-

4c. 登録フローの図解



5. コード解説

ここまで実装したコードの重要なポイントを、初心者向けに詳しく解説します。

5-1. Server Component と Client Component の使い分け

この章で作ったコンポーネントは、2種類に分かれています。

コンポーネント	種類	理由
<code>app/page.tsx</code>	Server Component	Supabase からのデータ取得をサーバー側で行うため
<code>StatusBadge</code>	Server Component	状態を持たない純粋な表示コンポーネントのため
<code>RatingStars</code>	Server Component	同上
<code>BookCard</code>	Server Component	同上
<code>BookList</code>	Server Component	同上
<code>LoadingSpinner</code>	Client Component	アニメーションの再利用性のため (<code>"use client"</code>)
<code>app/error.tsx</code>	Client Component	Next.js の仕様上必須 (<code>"use client"</code>)
<code>BookForm</code>	Client Component	<code>useState</code> , <code>onChange</code> 等を使うため (<code>"use client"</code>)
<code>app/books/new/page.tsx</code>	Client Component	<code>useRouter</code> , <code>useState</code> を使うため (<code>"use client"</code>)

なぜこの使い分けが重要なのか:

- **Server Component のメリット:** JavaScript バンドルに含まれないため、ブラウザに送信されるデータが少なく、ページの読み込みが速くなります。また、サーバー上で直接データベースにアクセスできるため、API エンドポイントを別途作る必要がありません。
- **Client Component のメリット:** `useState`、`useEffect`、イベントハンドラなどの React のインタラクティブ機能を使えます。ユーザーの操作に応じて画面を動的に更新できます。

初心者がつまづきやすいポイント:

「`"use client"` をどこに書くか迷う」— 原則として、`useState` や `useEffect`、`onClick` などのイベントハンドラを使うコンポーネントにだけ `"use client"` を付けます。書かなければ Server Component になります (App Router のデフォルト)。「迷ったら Server Component のままにして、エラーが出たら `"use client"` を追加する」というアプローチでも構いません。

5-2. statusConfig のパターン (StatusBadge)

```
const statusConfig: Record<BookStatus, { label: string; bgColor: string; textColor: string }> = {
  reading: { label: "読書中", bgColor: "bg-blue-100", textColor: "text-blue-800" },
  // ...
};
```

なぜ if 文ではなくオブジェクトを使うのか:

`if (status === "reading") { ... } else if (status === "completed") { ... }` と書くこともできますが、オブジェクトで管理する方が:

1. コードが短くなる: 条件分岐を書かなくてよい。
2. 拡張しやすい: 新しいステータスを追加するときにオブジェクトに1行追加するだけ。
3. 型安全: `Record<BookStatus, ...>` を使うことで、全てのステータスに対する設定を定義し忘れるとTypeScriptがエラーを出す。

5-3. `line-clamp-2` について (BookCard)

```
<h2 className="text-lg font-bold text-gray-900 line-clamp-2 flex-1">
```

`line-clamp-2` は Tailwind CSS のユーティリティクラスで、テキストを2行までに制限し、それを超える部分を `...` で省略します。書籍のタイトルが長い場合にカードのレイアウトが崩れるのを防ぎます。

5-4. Book 型と Supabase のテーブル構造の対応

```
export type Book = {  
  id: string;  
  title: string;  
  author: string;  
  publisher: string | null;  
  // ...  
};
```

なぜ `publisher` が `string | null` なのか:

データベースの `publisher` カラムは NULL を許容しています (任意入力項目だから)。TypeScript では `null` を明示的に型に含める必要があるため、`string | null` としています。

初心者がつまづきやすいポイント:

null と undefined の違い — データベースから取得した値が「存在しない」場合は `null` が返ります。一方、JavaScript のオブジェクトで「プロパティ自体が存在しない」場合は `undefined` になります。Supabase から返ってくるデータは常に `null` (`undefined` ではない) なので、型定義は `string | null` とします。

5-5. フォームの状態管理 (BookForm)

```
const [formData, setFormData] = useState<BookFormData>(initialData ?? defaultFormData);
```

なぜ1つの state にまとめるのか:

各フィールドごとに別々の `useState` を使う方法もあります:

```
// NG ではないが、フィールド数が多いと管理が大変
const [title, setTitle] = useState("");
const [author, setAuthor] = useState("");
const [publisher, setPublisher] = useState("");
// ...7つの useState...
```

これだと、フィールドが増えるたびに `useState` が増え、`handleChange` も個別に書く必要があります。1つのオブジェクトにまとめることで:

1. コードが簡潔: `handleChange` を全フィールドで共有できる。
2. データの受け渡しが楽: `onSubmit(formData)` で全フィールドを一度に渡せる。
3. 初期値の設定が楽: `initialData ?? defaultFormData` で一括設定できる（編集時に便利）。

5-6. `router.push` と `router.refresh` の組み合わせ（NewBookPage）

```
router.push("/"); // URLを変えて遷移
router.refresh(); // Server Component のデータを取り直す
```

なぜ両方必要なのか:

- `router.push("/")` はトップページに遷移しますが、Next.js はパフォーマンスのためにページの内容をキャッシュ（一時保存）しています。キャッシュが残っていると、登録前の古いデータが表示されることがあります。
- `router.refresh()` はキャッシュを無効化し、Server Component を再実行させます。これにより、Supabase から最新のデータが取得され、新しく登録した書籍が一覧に反映されます。これが冒頭で触れた **revalidation**（リバリデーション／再検証）の一つの形です。

補足: Server Action 方式なら `revalidatePath`: Server Action を使う場合は、`router.refresh()` の代わりにサーバー側で `revalidatePath("/")` を呼びます。意味は同じ「キャッシュを破棄して次の表示で取り直す」ですが、サーバー側からの指示なのでより確実です。

5-7. 空文字列を `null` に変換する理由

```
publisher: data.publisher.trim() || null,
```

なぜ空文字列のまま保存しないのか:

- データベースでは「値がない」ことを `NULL` で表現するのが一般的です。

- 空文字列 `""` と `NULL` は意味が異なります。`""` は「入力したが空だった」、`NULL` は「入力されていない」を意味します。
- 将来的に「出版社が未入力の書籍」を検索する場合、`WHERE publisher IS NULL` で統一的に取得できます。`""` が混在すると `WHERE publisher IS NULL OR publisher = ''` と書く必要があり面倒です。
- `data.publisher.trim() || null` は「空白を除去した後に空文字列なら null にする」という意味です。`||` 演算子は左辺が falsy（空文字列は falsy）なら右辺を返します。

5-8. e.preventDefault() の役割

```
const handleSubmit = async (e: FormEvent) => {
  e.preventDefault(); // ブラウザの「フォーム送信→画面遷移」を止める
  // ...
};
```

HTML の `<form>` は、送信ボタンが押されるとデフォルトでページ全体をリロードしてデータを送信しようとします（昔ながらの「サーバーへPOSTしてHTMLを返してもらおう」挙動）。`e.preventDefault()` はこのデフォルト動作を止め、JavaScript で非同期にデータを送信できるようにします。これを忘れるとページがリロードされ、`useState` の値がすべてリセットされてしまいます。

5-9. FormData と FormDataEntryValue について（参考）

この章ではブラウザ用 Supabase クライアントで `useState` の値を直接 INSERT していますが、Server Action 方式（参考コラム参照）では `formData.get("title")` のようにフォームから値を取り出します。その時の戻り値の型に注意が必要です。

```
// formData.get の戻り値の型
const value: FormDataEntryValue | null = formData.get("title");
//          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
//          FormDataEntryValue = string | File
//          フィールドが存在しないと null
```

- `FormDataEntryValue`: 「文字列か File（ファイル）か」のどちらか。普通の `<input type="text">` なら `string`、`<input type="file">` なら `File`。
- `null`: そもそもそのフィールドが送信されなかった場合（HTML 上に無い、disabled だった、など）。

そのため Server Action では `String(formData.get("title") ?? "")` のように、まず `??` で `null` を空文字に変換し、その後 `String()` で文字列にキャストするのが安全です。

```
const title = String(formData.get("title") ?? "");
//          ^^^^^          null だったら "" を使う
//          ^^^^^          string | File を string に変換
```

5-10. Server Action のセキュリティ（参考）

Server Action は「サーバー上で実行される関数」です。クライアント（ブラウザ）から呼び出せますが、コードはサーバーで走るため:

- シークレット情報を扱える: 環境変数の `SUPABASE_SERVICE_ROLE_KEY` など、ブラウザに漏らしたくない情報をそのまま使える。
- データベースに直接アクセスできる: クライアントを経由しないので、改ざんされにくい。
- ただしクライアントからの入力信用しない: フォームから送られた `formData` の中身は誰でも書き換えられる可能性があるため、サーバー側でも改めてバリデーションする必要がある。

覚えておくべきこと:「クライアントから来た値はすべて疑え」は Web 開発全般の鉄則です。Client Component 方式（本書）でも、Supabase の Row Level Security（RLS）ポリシーが「最後の砦」として働きます。

6. 動作確認手順

以下の手順で、実装した機能が正しく動作するか確認しましょう。

前提

- 開発サーバーが起動していること（`npm run dev` を実行済み）
- Supabase のプロジェクトとデータベースが前章で構築済みであること
- `.env.local` に Supabase の接続情報が設定されていること

ステップ1: トップページが表示確認（書籍0冊の状態）

1. ブラウザで `http://localhost:3000` を開きます。
2. 以下を確認してください: - ヘッダーに「書籍管理アプリ」と表示されている - 「+ 新規登録」ボタンが右上に表示されている - 「全 0 冊」と表示されている - 「書籍が登録されていません」というメッセージと「最初の書籍を登録する」ボタンが中央に表示されている
3. ブラウザの開発者ツール（F12）の Console タブにエラーが出ていないことを確認します。

ステップ2: 登録ページへの遷移確認

1. ヘッダーの「+ 新規登録」ボタンをクリックします。
2. `http://localhost:3000/books/new` に遷移し、登録フォームが表示されることを確認します。
3. 以下の要素が表示されていることを確認: - 左上に戻るボタン（矢印アイコン） - 「書籍を登録する」というタイトル - タイトル、著者、出版社、出版日、評価、ステータス、メモの各フィールド - 「登録する」と「キャンセル」ボタン

ステップ3: バリデーションの確認

1. 何も入力せずに「登録する」ボタンをクリックします。
2. 「タイトルは必須です」「著者は必須です」というエラーメッセージが赤文字で表示されることを確認します。
3. タイトルと著者のフィールドの枠線が赤くなっていることを確認します。
4. タイトルフィールドに何か入力すると、タイトルのエラーメッセージが消えることを確認します。

ステップ4: 書籍の登録

1. 以下のテスト用データを入力します: - タイトル: **リーダブルコード** - 著者: **Dustin Boswell** - 出版社: **オライリージャパン** - 出版日: **2012-06-23** - 評価: **★★★★☆ (4)** - ステータス: **読了** - メモ: **読みやすいコードを書くための実践的なテクニック集。**
2. 「登録する」ボタンをクリックします。
3. ボタンが一瞬「送信中...」に変わることを確認します。
4. トップページ (<http://localhost:3000>) にリダイレクトされることを確認します。
5. 「全 1 冊」と表示され、登録した書籍のカードが表示されることを確認します。

ステップ5: カード表示の確認

1. 表示されたカードに以下の情報があることを確認: - タイトル「リーダブルコード」 - ステータスバッジ「読了」（緑色） - 著者「Dustin Boswell」 - 出版社「オライリージャパン」 - 出版日「2012-06-23」 - 星評価「★★★★☆ (4)」 - メモ「読みやすいコードを書くための実践的なテクニック集。」
2. カードにマウスを乗せると影が変化することを確認します。

ステップ6: 複数冊の登録とグリッド表示の確認

1. さらに2〜3冊の書籍を登録します（ステータスを変えて登録すると、バッジの色分けも確認できます）。
2. 画面幅を変えて、グリッドレイアウトのレスポンス動作を確認します: - デスクトップ（1024px 以上）: 3列 - タブレット（768px〜1023px）: 2列 - スマホ（767px 以下）: 1列

ステップ7: Supabase ダッシュボードでのデータ確認

1. Supabase ダッシュボード (<https://supabase.com/dashboard>) にアクセスします。
 2. Table Editor で **books** テーブルを開きます。
 3. 登録した書籍のレコードが正しく保存されていることを確認します。
-

7. トラブルシューティング

問題1: データが表示されない（一覧が常に空）

症状: 書籍を登録したはずなのに、トップページに書籍が表示されない。「全 0 冊」のまま。

原因と対策:

1. Supabase の接続情報が正しくない

`.env.local` ファイルの内容を確認してください。

```
bash # .env.local NEXT_PUBLIC_SUPABASE_URL=https://xxxxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbG...
```

- URL の末尾にスラッシュ `/` が余計についていないか確認してください。
- `.env.local` を変更した場合は、開発サーバー（`npm run dev`）を再起動する必要があります（`Ctrl+C` で停止し、再度 `npm run dev`）。

2. Row Level Security (RLS) の設定

Supabase のテーブルに RLS が有効になっていると、ポリシーが設定されていない場合はデータを取得できません。

Supabase ダッシュボードで確認: - Authentication → Policies → `books` テーブル - SELECT に対するポリシーが設定されているか確認 - テスト段階では一時的に RLS を無効にすることもできます（ダッシュボードの Table Editor → `books` テーブル → RLS を OFF に）

3. Supabase クライアントのインポートパスが間違っている

`app/page.tsx`（Server Component）では `@/lib/supabase/server` からインポートし、
`app/books/new/page.tsx`（Client Component）では `@/lib/supabase/client` からインポートする必要があります。逆にするとエラーになります。

4. ブラウザの開発者ツールで確認する

F12 → Console タブを開き、エラーメッセージがないか確認してください。また、Network タブで Supabase への API リクエストが成功しているか（ステータスコード 200）を確認してください。

問題2: フォーム送信でエラーが出る

症状: 「登録する」ボタンを押すと、エラーメッセージ（「書籍の登録に失敗しました」）が表示される。

原因と対策:

1. テーブルのカラム名が一致していない

`insert` に渡すオブジェクトのキー名が、データベースのカラム名と一致していない可能性があります。ブラウザの Console にエラー詳細が表示されているはずなので確認してください。

```
// よくある間違い publishedDate: "2024-01-01" // NG: キヤメルケース published_date: "2024-01-01" // OK: スネークケース (DB のカラム名に合わせる)
```

2. NOT NULL 制約違反

データベースの `title` や `author` カラムに NOT NULL 制約がかかっている場合、空文字列や `null` を送信するとエラーになります。BookForm のバリデーションで必須チェックを行っていますが、バリデーションをバイパスして送信した場合に発生します。

3. RLS ポリシーの INSERT が許可されていない

Supabase のテーブルに RLS が有効になっていて、INSERT のポリシーが設定されていない場合、データを挿入できません。SELECT と同様に、INSERT のポリシーも設定する必要があります。

4. Supabase のクライアント用ライブラリを使っているか確認

`app/books/new/page.tsx` は Client Component なので、`@/lib/supabase/client` を使います。`@/lib/supabase/server` を使うと `cookies()` などのサーバー専用APIにアクセスしようとしてエラーになります。

問題3: 型エラーが出る

症状: TypeScript のコンパイルエラーが発生する。

原因と対策:

1. Book 型のインポート漏れ

```
Cannot find name 'Book'.
```

`Book` 型を使うファイルで、正しくインポートしているか確認してください。

```
tsx import { type Book } from "@/components/BookCard";
```

`type` キーワードを使っているのは「型だけをインポートする」という意味です。TypeScript 専用の構文で、コンパイル後の JavaScript からは削除されます。なくても動きますが、付けておくとインポートの意図が明確になります。

2. BookStatus 型のインポート漏れ

```
Cannot find name 'BookStatus'.
```

`StatusBadge.tsx` からエクスポートしている型なので:

```
tsx import { type BookStatus } from "@/components/StatusBadge";
```

3. Supabase の型定義との不一致

Supabase CLI で型を自動生成している場合、自動生成された型と手動で定義した `Book` 型に差異があるとエラーになることがあります。自動生成された型を使う方法は後の章で解説しますが、現段階では手動で定義した型で問題ありません。

4. `null` と `undefined` の混在

Type 'undefined' is not assignable to type 'string | null'.

Supabase から返るデータは `null` (`undefined` ではない) です。型定義で `string | undefined` ではなく `string | null` を使ってください。

5. `createClient` の名前衝突

サーバー用とクライアント用で同じ名前 `createClient` をインポートしていますが、ファイルが異なるので衝突しません。ただし、1つのファイルで両方使おうとすると衝突します。そのような場合はエイリアスを使います:

```
tsx import { createClient as createServerClient } from "@lib/supabase/server"; import {
createClient as createBrowserClient } from "@lib/supabase/client";
```

まとめ

この章では、以下の機能を実装しました。

- 一覧表示: Server Component でSupabase からデータを取得し、BookCard を使ってグリッド表示
- ステータス・評価の視覚化: StatusBadge (色分けバッジ)、RatingStars (星評価)
- ローディング状態: loading.tsx とスケルトン UI
- エラーハンドリング: error.tsx とリトライ機能
- 新規登録: BookForm でフォーム入力、バリデーション、Supabase への INSERT、リダイレクト

次の章では、**詳細表示 (Read)**、**編集 (Update)**、**削除 (Delete)** を実装し、CRUD の残りの機能を完成させます。